Техническая спецификация: платформа SCORM-шаблонов

Статус: контракт формата шаблонов — источник истины для внутренних и внешних SCORM-шаблонов. Описывает действующее устройство: браузерный рендер `shell` и макетов, единый манифест, path-only DSL, `resultField` с рендерерами `core.*`, подгонку текста `textFit`, структурную валидацию и браузерную smoke-проверку, системные узлы раздела (`review`, `section-results`), варианты стартового экрана (`kind: "start"` со своим `layoutFile` и свойством страницы `image`) и документ отчёта о результатах: оболочку (`kind: "report"` / `report.adaptive`, §8.4) и блоки (`kind: "report.block"`, `reportDocument`, §8.4.7-8.4.9) с постраничной раскладкой и принудительным разрывом листа (§8.4.6). За рамками формата — логика прохождения (задаётся сценарием теста, `flowPolicy`) и вычисляемые результаты (шкалы и показатели, публикуются в контекст рендера): см. §13 и §14. Настраиваемые надписи интерфейса итогов и порядок их подблоков — §8.3.1-8.3.3 и §10.5, разрез результата в карточке темы и окраска его полос — §10.6, блоки разделов — §10.7, сводный разрез по тесту — §10.8, надпись лимита времени — §10.9, толкование темы и подтемы — §10.10, заголовки итога — §10.11, условие прохождения на обложке и во введении раздела — §10.12, порог в баллах на обложке и тумблеры её плиток — §10.13. Ещё не действующие возможности вынесены в §3.
Версия: 3.17.0 (см. «История версий»)
Дата актуализации: 2026-11-02

История версий

Версия документа отражает состояние КОНТРАКТА формата шаблонов и версионируется семантически (`MAJOR.MINOR.PATCH`) — **отдельно** от `templateApiVersion` в манифесте (это версия рантайм-API шаблона, сейчас `1.0`, §5). Правила бампа:

- **MAJOR** — несовместимое изменение формата: ранее валидные шаблоны требуют правок (удалено/переименовано поле манифеста, изменён контракт слота, убран тип поля).
- **MINOR** — обратно-совместимое расширение: новый тип экрана, поля, параметра или настройки; прежние шаблоны продолжают работать без изменений.
- **PATCH** — уточнения формулировок, примеры, исправления опечаток без изменения контракта.

Версия	Дата	Изменения
3.17.0	2026-11-02	**Порог в баллах на обложке и тумблеры плиток обложки** (§10.13). Новое необязательное поле `course.passPoints` (`value` , `valueLabel` , `label`): общий порог, заданный в БАЛЛАХ, обложка называет в баллах — «5 / баллов для прохождения». `course.passPercent` у такого теста пуст: постоянного процента у порога в баллах нет, максимум зависит от выданных вопросов и их цены. Прежде пакет делил порог на число вопросов и печатал, например, «125 %», а тот же процент уходил в LMS порогом согласования балла с вердиктом; теперь пакет пересчитывает порог по возможным баллам каждой попытки (`lmsThresholdFor`). Варианты вида `start` объявляют четыре тумблера страницы — `questionCountShown` , `passConditionShown` , `timeLimitShown` , `maxAttemptsShown` (`boolean` , умолчание `true`): выключенный тумблер ядро разрешает в пустое поле `course.*` , и плитку убирает её же гейт. Версия МИНОРНАЯ: тест с порогом в процентах и страница без сохранённых значений дают контекст байт в байт прежний; шаблон, не связывающий `passPoints` , порог в баллах просто не показывает, как веб и до этого. Плитка и тумблеры добавлены во все три поставляемых шаблона.
3.16.1	2026-10-31	**Строку числа попыток в отчёте гасит автор** (§10.2, уточнение). Настройка теста «Показывать число попыток» (`tests.show_attempts_count`) отдаёт `report.attemptsCountLabel` пустой строкой, и гейт `{{#if report.attemptsCountLabel}}` убирает строку «Лучший результат за N попыток» из шапки документа. Прежде подпись пустела только у теста без вердикта. Версия ПАТЧ: поля контекста те же, а требование гейтить строку на пустоте уже было в контракте; все поставляемые шаблоны его выполняют.
3.16.0	2026-10-16	**Картинки на карточках хаба — свойство варианта маршрутизатора** (§8.1, §8.2.1). Новый тип настройки `itemImages` : карта «ключ пункта → картинка», где ключ — `topic:<id>` темы или `scenario:<id>` пункта-сценария. Вариант вида `router` объявляет её под ключом `cardImages` ; объявивший вариант показывает картинки, остальные — нет, поэтому шаблон, которому они нужны, добавляет ВТОРОЙ вариант маршрутизатора («с картинками»), а автор выбирает между ними «Сменить макет». Разметка карточки: картинка стала собственным дочерним элементом `router-topic-card__img` (раньше лежала внутри `router-topic-card__top`), карточка с картинкой несёт модификатор `router-topic-card--with-img` ; без картинки карточка прежняя. Поставляемые шаблоны переписали правило `__img` под новое место. Версия МИНОР: новый тип настройки и новый модификатор; прежние шаблоны работают — картинок у них нет, и карточка без картинки не изменилась.
3.15.1	2026-10-08	**Вопрос «Сценарий» и карточка сценария в хабе — шаблону ничего не нужно** (§8.1, уточнение). Трек «Сценарий в ИС» добавил тип вопроса, обложку и окно правил в области ответа (`question-interaction`), плеер на весь экран и пункт-сценарий роутера. Всё это рисует ядро встроенными стилями и токенами дизайн-системы; манифест, макеты, слоты, контекст рендера и `theme.css` не менялись. Новые экспорты глобала `TBTemplate` в пакете (`renderSimCover` , `renderSimRulesDialog` , `simCoverState` , `simCoverShot` , `simRules` , `simRunReplaces` , `mountPlayer` , `simulationRatio` , `encodeSimProtocol` , `simProtocolChunks`) — внутренний слой ядра, не часть формата. Версия ПАТЧ: контракт не расширен, записано только то, чего от шаблона НЕ требуется.

Версия	Дата	Изменения
3.15.0	2026-10-06	**Условие прохождения на введении раздела гасит автор** (§10.12). Вариант вида `intro` объявляет тумблер `passConditionShown` (`boolean` , умолчание `true` , «Показывать условие прохождения»). Выключенный, он отдаёт `sectionIntro.passCondition` пустой строкой, и гейт `{{#if sectionIntro.passCondition}}` убирает строку «Для прохождения: …». Это настройка СТРАНИЦЫ: каждое введение раздела решает за себя. Порог темы, вердикт, `isRequired` и `timerWarning` от тумблера не зависят. Версия МИНОРНАЯ и обратно-совместимая: макеты не менялись, страница без сохранённого значения и вариант, не объявивший настройку, печатают строку как раньше. Тумблер объявлен у всех трёх поставляемых шаблонов.
3.14.0	2026-09-30	**Условие прохождения до первого ответа** (§10.12). Обложка получает `course.passCondition` (`count` , `total` , `kind` , `countLabel` , `label`), когда исход решают темы: «8 из 8 — обязательных тем для прохождения» или «5 из 5 — тем для прохождения». Введение раздела получает `sectionIntro.passCondition` (порог темы по выданному варианту: «Для прохождения: 70 %» / «Для прохождения: 7 баллов из 10»), `sectionIntro.isRequired` (метка «Обязательная тема» там, где обязательность решает вердикт) и `sectionIntro.timerWarning` (системное предупреждение о старте отсчёта раздела). Формулировки строит один модуль ядра по правилам судьи. Версия МИНОРНАЯ и поля НЕОБЯЗАТЕЛЬНЫЕ: у теста под политикой «Только общий результат» контекст обложки байт в байт прежний, а шаблон, новых полей не связывающий, работает как работал. Все три поставляемых шаблона переведены.
3.13.0	2026-09-24	**Признак «выход закрывает раздел»** (§10.9, PRD-67). В контекст стартового экрана добавлено поле `course.closesOnLeave` : `true` , когда автор включил настройку «Закрывать раздел при выходе» и у теста есть хоть один лимит времени. Выход из начатого раздела тогда закрывает его навсегда, а в тесте без разделов завершает попытку. Макет может предупредить ученика ДО начала; поставляемые шаблоны признак не печатают — оба хоста предупреждают уведомлениями ядра. Версия МИНОРНАЯ и поле НЕОБЯЗАТЕЛЬНОЕ: у теста без настройки его в контексте нет, контекст байт в байт прежний.
3.12.0	2026-09-24	**Окраска полос подтем — настройка автора, заливку считает ядро** (§6, §10.6, §10.8). Параметр оформления `breakdownBarFill` выбирает, что кодирует цвет полосы: исход (`verdict` , прежнее поведение), долю (`share`) или ничего (`neutral`). В режиме «По доле» строка разреза получает готовую заливку `barFill` — прямой градиент между цветами схемы уровней теста до цвета в точке доли — и пустой `passClass` ; макет печатает её в `style` полосы. Параметр один на экран итогов и отчёт, поэтому они больше не расходятся. Там же `default` манифеста начал действовать для параметров, которые ядро читает само (схема уровней, вид шкал и показателей, окраска полос): шаблон задаёт вид нетронутого теста, а редактор показывает применяемое значение. Версия МИНОРНАЯ: без параметра строка байт в байт прежняя, а у поставляемых шаблонов умолчания манифеста и ядра совпадали. Все три поставляемых шаблона печатают `barFill` ; «Сертификация» по умолчанию окрашивает полосы по доле в брендовой паре цветов, а её приём с `background-size` в `report.css` снят.
3.11.0	2026-09-24	**Ссылка `<a href>` в отчёте нажимается и в PDF** (§8.4.5). Конвейер экспорта накладывает настоящие ссылки PDF поверх растра не только по чипам `.pdf-link-btn[data-url]` , но и по каждой `<a href>` с адресом `http(s)://` или `mailto:` —

Версия	Дата	Изменения
		прежде всего по ссылкам, которые автор вставил в текст страницы документа, вводного блока или рекомендаций. Область строится по строкам ссылки (`getClientRects`), поэтому перенесённая ссылка нажимается на каждой строке и не захватывает соседний текст; якорь `#…` и относительный путь не переносятся. До этого такая ссылка в PDF выглядела ссылкой и не нажималась. Версия МИНОРНАЯ: от шаблона ничего не требуется — авторский `richText` , напечатанный как есть, получает рабочие ссылки сам, а контракт `.pdf-link-btn` действует по-прежнему.
3.10.0	2026-09-21	**Толкование темы и подтемы, заголовки итога** (§10.10, §10.11). В контекст строки темы добавлены `interpretationHtml` — готовая разметка текста, который ОБЪЯСНЯЕТ результат, — и такое же поле у строки разреза; рядом два признака раскладки, `hasBreakdownNotes` и `hasAside` , которые ядро считает за макет (движок не умеет ни перебрать строки, ни сложить два вопроса в один ответ, §9). Толкование отличается от обратной связи ПРАВИЛОМ ВЫДАЧИ: оно печатается при любом вердикте, стоит при своей теме или полосе и в сводный блок «Рекомендации» не попадает. Отдельно вариант `results.*` получил три настройки заголовков итога (`headingDocument` / `headingPassed` / `headingFailed`): тест называет свой итог сам, а НОВЫХ полей контекста при этом не появляется — значения приходят в прежние `course.title` , `result.statusLabel` и `report.verdictHeadline` . Версия МИНОРНАЯ, все поля НЕОБЯЗАТЕЛЬНЫЕ: тест, ничего не заполнивший, даёт контекст байт в байт прежний, а шаблон, новых полей не связывающий, работает как работал. Все три поставляемых шаблона переведены.
3.9.0	2026-09-21	**Подзаголовок раздела настраивает АВТОР ТЕСТА, а не вёрстка** (§10.1, PRD-22 §11). Строка над авторской инструкцией на экране «Введение раздела» была жёстким словом «Инструкция» в макете. Вариант вида `intro` объявляет её двумя обычными настройками страницы — тумблером `sectionSubtitleShown` (`boolean` , умолчание `true`) и формулировкой `sectionSubtitle` (`text` , умолчание «Инструкция»), — а ядро отдаёт разрешённую строку макету как `page.sectionSubtitle` в том же неймспейсе, где живёт `page.nextLabel` . Гасит надпись ТОЛЬКО тумблер: пустое поле означает «следовать шаблону» и печатает умолчание, поэтому макет проверяет строку на непустоту (`{{#if page.sectionSubtitle}}`), а не печатает её безусловно. Расширение обратно-совместимое: вариант, не объявивший настроек, печатает то, что стоит в его макете, а страница без сохранённых значений — умолчание шаблона.
3.8.0	2026-09-20	**Контекст итогов несёт признак «часть ответов на проверке»** (§7.3, PRD-57 FR-41). У `result` появилось поле `pendingReview` : `true` , когда в попытке есть ответ, который никто не проверял (развёрнутый ответ §5; ответ, чьё выражение не уложилось в бюджет §6.4). Текст и место — за шаблоном: контракт несёт ФАКТ, а не формулировку. Поле заводится ДО появления самой проверки намеренно — добавить его позже значило бы поднять версию ещё раз и обновить внешние шаблоны из реестра PRD-3. Версия МИНОРНАЯ и поле НЕОБЯЗАТЕЛЬНОЕ: у теста без открытых заданий контекст остаётся байт в байт прежним, а шаблон, ничего не знающий о признаке, работает как работал.
3.7.0	2026-09-19	**Слот текста задания перестаёт быть заголовком** (§8, PRD-57 FR-03d). Тег `<h2 class="tb-scene__qtitle" data-slot="question-text">` заменён на `<div class="tb-scene__qtitle" role="heading" aria-level="2" data-slot="question-text">` . Причина в HTML, а не во вкусе: в заголовке допустим только

Версия	Дата	Изменения
		строчный контент, поэтому блочный листинг кода и блочная формула туда невалидны — а именно их приносят PRD-57 §4.1 и §4.2. Вид на экране не меняется, роль для экранного диктора сохраняется явно. Версия МИНОРНАЯ: имя слота, его наполнение и контекст прежние, и шаблон, оставшийся с `<h2>`, продолжает работать — он просто не покажет блочный листинг. Все три поставляемых шаблона переведены.
3.6.0	2026-09-18	**Лимит времени приходит готовой надписью** (§10.9). Рядом с прежним `course.timeLimitMinutes` в контекст добавлено поле `course.timeLimitLabel` — те же минуты, разложенные ядром в дни, часы и минуты («14 дней», «2 ч 30 мин», «45 мин»); той же надписью печатается лимит раздела (`sectionIntro.timeLimitLabel`). Единица измерения переехала из макета в ядро, потому что движок намеренно без выражений и хелперов (§9): макет с зашитым «мин» на двухнедельном бюджете говорил «20160 мин», а его отсчёт — «20160:00». Рантайм обоих хостов печатает отсчёт теми же единицами: `M:SS`, `H:MM:SS` от часа, `D д H:MM:SS` от суток. Версия МИНОРНАЯ: число осталось в контексте, и шаблон, связывающий только его, работает как прежде. Оба поставляемых шаблона переведены на надпись — узел печатает её через `data-path`, а гейт блока стоит на ней же.
3.5.0	2026-09-18	**Внешняя ссылка в авторском тексте сохраняется** (§8.2.1.1). Правило «внешний `http(s)://` в `src` / `href` удаляется целиком» разделено по смыслу: `src` называет ресурс, который страница грузит сама, и остаётся запрещённым везде, а `href` у `<a>` / `<area>` — направление, по которому учащийся идёт сам, и до клика не запрашивает ничего. Такой адрес теперь СОХРАНЯЕТСЯ и получает `target="_blank" rel="noopener noreferrer"`, чтобы переход не увёл кадр с незавершённой попытки; `href` у прочих тегов (в том числе `<base>`) снимается по-прежнему. Прежде панель форматирования предлагала кнопку «Ссылка» (PRD-22 FR-33), а сохранялся `<a>` без адреса — от простого текста не отличить. Версия МИНОРНАЯ: ранее валидный шаблон продолжает работать, но обязан ОФОРМИТЬ ссылку — у `<a>` без адреса браузерного умолчания не было, и правила для неё в шаблоне могло не быть. Оба поставляемых шаблона правило уже несут: области авторской разметки в `theme.css` и общее `.tb-report a` в `report.css`.
3.4.0	2026-09-18	**Описание теста печатается с оформлением** (PRD-59 FR-11). В контекст стартового экрана добавлено парное поле `course.descriptionHtml` рядом с прежним `course.description` (§10.4). Ядро строит его из текста и его формата (`tests.description_format`) по той же политике, что и остальной авторский текст: `plain` экранируется и переводы строк становятся ` `, `richText` и `html` печатаются как есть. Гейт блока по-прежнему стоит на СТРОКЕ, поэтому пустое описание не печатает ничего. Версия МИНОРНАЯ: шаблон, связывающий только строку, работает как прежде и показывает описание без оформления. Оба поставляемых шаблона правку уже несут — узел описания стал блочным и печатается через `{{& … }}` БЕЗ `data-path`: проход по `[data-path]` заполняет узел через `textContent` после прохода DSL и затёр бы разметку.
3.3.0	2026-09-11	**Превью варианта умеет различать тему редактора** (§6): значением `optionPreviews` может быть не только путь, но и пара `{ light, dark }`. Лок, нарисованный под светлый фон, на тёмном редакторе нечитаем, а комплект фирменного блока обычно содержит оба варианта — теперь карточка показывает тот, что подходит теме, в которой

Версия	Дата	Изменения
		сидит автор. Версия МИНОРНАЯ: строка по-прежнему допустима и означает одну картинку на обе темы. Значение вне этих двух форм блокирует импорт.
3.2.0	2026-09-11	**Параметр умеет показать варианты картинками и попасть в атрибут сцены** (§6). `optionPreviews` — путь к файлу пакета на каждый вариант `select`: редактор рисует карточки с изображениями вместо выпадающего списка, потому что выбирают ОБЛИК, а словом его не описать. `dataAttr` — имя атрибута вида `data-имя`, в который оба хоста кладут выбранное значение на корне сцены: кастомное свойство — только значение, по нему нельзя ВЫБРАТЬ правило, а шаблон, меняющий картинку по варианту, выбирает селектором. Версия МИНОРНАЯ: оба поля необязательны, шаблон без них работает как прежде — `select` остаётся списком, атрибут не ставится. Кривое имя атрибута и отсутствующий файл превью блокируют импорт, а не всплывают молчаливой пустой карточкой при прохождении.
3.1.0	2026-09-05	**Строка разреза снова несёт исход подтемы** (PRD-50 §16, FR-54/FR-57): в контекст строки возвращены `passed` и `passClass`, к ним добавлена `requiredLabel` — готовая надпись порога («Нужно 70 %»), которая приходит только там, где автор включил показ значения (§10.6). Отличие от снятого версией 3.0.0 набора принципиальное: СЛОВЕСНОЙ метки исхода (`statusLabel`) у строки нет и не будет — строка узкая, слово рядом с процентом дублировало бы цвет, а вердикт СЛОВОМ говорит карточка темы вокруг. Исход берётся у записи разреза, а не вычисляется раскладкой, поэтому старая попытка не перекрашивается при смене настроек теста. Версия МИНОРНАЯ: шаблон, ничего о новых полях не знающий, печатает ровно то, что печатал — класс подставляется в атрибут пустой строкой, надпись гейтится своим `{{#if requiredLabel}}`.
3.0.0	2026-09-03	**Строка разреза больше не несёт вердикта** (решение владельца 2026-09-03): из контекста строки убраны `passed`, `passClass` и `statusLabel` (§10.6), заведённые версией 1.7.0. Подтема считается и показывается, но не судится — она говорит о результате, а вердикт выносит ТЕМА, чья карточка стоит вокруг строки. Версия МАЖОРНАЯ по правилу бампа: из контракта УДАЛЕНЫ поля, и шаблон, который их связывал, требует правки раскладки. Прежние шаблоны при этом не ломаются: класс выходит пустым, а гейт `{{#if statusLabel}}` — всегда ложным, поэтому лишняя разметка просто не печатается. Оба поставляемых шаблона правку уже несут; порог ключа раздела продолжает существовать и участвовать в вердикте ТЕМЫ — снят только его показ в строке.
2.0.0	2026-08-16	Отчёт стал ДОКУМЕНТОМ ИЗ БЛОКОВ (PRD-51). **Оболочка:** вид `report` / `report.adaptive` объявляет корень документа, а не всю страницу; блоки движок вкладывает её прямыми детьми, и контейнер между корнем и блоком ломает постраничную раскладку (§8.4.1). **Блоки:** новый вид `report.block` с обязательным ключом блока из ЗАКРЫТОГО перечня продукта, необязательным `kinds[]` и умолчанием на пару «блок + вид» (§8.4.7). **Документ по умолчанию:** корневой ключ `reportDocument` по видам; `page-break` в нём вправе повторяться, блок с данными — нет (§8.4.8). **Поля содержимого:** запрет `placeholders[]` снят с блока `page` — авторская страница документа несёт содержимое, которое пишет автор теста; у оболочки запрет остаётся. **Надписи:** перечень для отчёта сканируется и по макетам блоков (§8.3.2). Версия МАЖОРНАЯ при том, что прежние шаблоны печатаются как раньше (§8.4.9):

Версия	Дата	Изменения
		изменился СМЫСЛ вида `report`, и разработчик, читавший контракт 1.x, сделает о нём неверный вывод — завернёт блоки в свой контейнер и потеряет разбивку на листы.
1.7.0	2026-08-15	Три расширения одного трека (PRD-50, этапы Э2-Э5). **Вердикт ключа:** строка разреза несёт `passed` / `passClass` / `statusLabel` (§10.6) — `null` и пустые строки, когда порога у ключа нет или ключ не попал в выданное, поэтому строка никогда не объявляет провал, которого гейт не выносил. **Блоки разделов:** `result.topicGroups[]` со счётчиком «пройдено N из M» и `result.ungroupedTopics[]` (§10.7); плоский `topicResults` остаётся полным рядом с ними, поэтому шаблон печатает либо блоки, либо список. **Сводный разрез по тесту:** `result.breakdown[]` — одна строка на ключ по всей выдаче — приезжает пятым подблоком итогов (`breakdown` в `resultsBlockOrder` и `result.blocks[].isBreakdown`, §8.3.3 и §10.8), печатается и на адаптивных итогах, где карточка темы полос не несёт. Все три обратно-совместимы: шаблон, не печатающий новых полей, работает как прежде.
1.6.0	2026-08-14	Два расширения и одно уточнение. **Разрез результата (PRD-50):** строка темы несёт готовые подытоги по ключам оси — `result.topicResults[].breakdown[]` (§10.6): ключ, ширина полосы и подпись значения приходят посчитанными, показ включает автор теста настройкой содержания, а вердикт от неё не зависит. **Принудительный разрыв страницы отчёта:** пустой узел с атрибутом `data-page-break` в макете приказывает постраничной раскладке начать новый лист (§8.4.6). **Уточнение к надписям:** перечень надписей, которые редактор предлагает для ОТЧЁТА, считается сканированием макетов вариантов отчёта, а не объявляется в манифесте вторым списком (§8.3.2). Оба расширения обратно-совместимы: шаблон, не печатающий разрез и не ставящий метку разрыва, работает как прежде.
1.5.0	2026-08-12	Надписи интерфейса итогов объявляет ШАБЛОН, а не зашивает в макет (PRD-49): раздел манифеста `labels[]` (§8.3.1) с умолчаниями по экранам и хранением значения теста как ЗАПИСИ с тремя состояниями (§8.3.2); порядок и состав четырёх подблоков итогов — `resultsBlockOrder` по экранам (§8.3.3); контекст несёт разрешённое дерево `labels.*` и уже собранный массив подблоков `result.blocks[]` (§10.5). Расширение обратно-совместимое: шаблон, не объявивший `labels[]`, печатает собственные жёсткие строки макета, как и раньше.
1.4.0	2026-08-09	Контекст несёт ВВОДНЫЙ БЛОК `result.introHtml` (§10.3) — авторский текст, который печатается первым на экране итогов и в отчёте; тексты этих двух выдач задаются раздельно. Расширение обратно-совместимое: макет, не печатающий поле, работает как прежде.
1.3.0	2026-08-09	Поле варианта отчёта объявляет НАЗНАЧЕНИЕ — `settings[].scope` : `content` (что попадёт в документ; правится в «Настройках», рядом с обратной связью) или `appearance` (как документ выглядит; правится в «Оформлении») — см. §8.4.1. Расширение обратно-совместимое: умолчание `appearance` оставляет поля шаблона, не знающего о признаке, ровно там, где автор их находил.
1.2.0	2026-08-09	Форматирование авторского текста доведено до выдачи. В DSL добавлена интерполяция контролируемого HTML `{{& path }}` — единственный канал разметки внутри `{{#each}}` (§9); `{{{ path }}}` остаётся запрещённым. Контекст несёт разметку парными полями `textHtml` / `textsHtml` рядом с прежними строками (§10.3): ядро

Версия	Дата	Изменения
		строит их из текста и его формата (`plain` экранируется, переводы строк становятся ` ` ; `richText` и `html` печатаются как есть). Расширение обратно-совместимое: шаблон, печатающий только строку, работает как прежде, но показывает текст без форматирования.
1.1.0	2026-07-31	Отчёт о результатах вошёл в формат шаблона (PRD-27): виды `report` и `report.adaptive` в `contentTemplates[]` со своим `layoutFile` , необязательным `styleFile` и `settings[]` (§8.4); картинки отчёта (подложка, логотип) — файлы ШАБЛОНА, объявленные полями типа `image` (§8.4.1.1); контекст отчёта `report.*` (§10.2); среда стилей отчёта — контейнер вне сцены, корневой класс `tb-report` , скоупленный CSS (§8.4.3); статические проверки объявления и файлов варианта (§17.1, §17.2) и рендер отчёта в smoke-проверке (§17.3); хранение выбора варианта и значений полей — вне `design_settings_json` (§16). Расширение обратно-совместимое: шаблон, не объявивший вида отчёта, деградирует на макет стандартного шаблона.
1.0.0	2026-07-28	Первая версионированная редакция. Зафиксировано текущее состояние контракта: манифест и обязательные поля (§5), path-only DSL (§9), `resultField` с рендерерами `core.*` и `textFit` (§8.2.1.x), разделение `placeholders[]` / `settings[]` (§8.2.1), системные узлы раздела `review` / `section-results` (§8.2), две палитры и `themes[]` (§6), варианты стартового экрана (`kind: "start"` со своим `layoutFile`) и параметр иллюстрации `startImageUrl` (§8.2).

---

1. Назначение

Этот документ определяет общий технический контракт для внутренних и внешних SCORM-шаблонов. Он является источником истины для формата шаблонов: прочие документы опираются на него, а не переопределяют механику шаблонов независимо.

Платформа поддерживает:

- единую механику для встроенных и загружаемых шаблонов;
- исполнение шаблона только в браузере внутри сгенерированного SCORM-пакета;
- полную настройку макетов страниц, а не только цветов и шрифтов;
- контролируемую ответственность Core за оценку, состояние навигации, SCORM-состояние и стандартные интерактивы вопросов;
- расширение через `template.js` и Runtime API.

---

2. Базовые принципы

2.1 Исполнение только в браузере

Шаблоны исполняются только в браузере обучающегося внутри SCORM-пакета.

Сервер не выполняет HTML или JavaScript шаблона во время экспорта. Сервер только:

- валидирует ZIP загруженного шаблона;
- хранит метаданные шаблона;
- копирует выбранный шаблон в сгенерированный SCORM ZIP;
- добавляет в пакет данные теста и ресурсы Core runtime.

2.2 Единая механика внутренних и внешних шаблонов

Встроенные и внешние шаблоны используют одинаковые:

- структуру `manifest.json`;
- структуру ZIP/файлов;
- браузерный рендерер;
- контракт макетов;
- Runtime API;
- процесс валидации и smoke-проверки.

Физическое хранение может отличаться, но runtime-поведение одинаково.

2.3 Ответственность Core и шаблона

Core владеет:

- состоянием теста;
- состоянием навигации;
- состоянием ответов;
- оценкой;
- состоянием обратной связи;
- восстановлением сессии;
- интеграцией с SCORM API;
- telemetry;
- рендерерами стандартных интерактивов вопросов;
- финальными защитами для критичных действий.

Шаблон владеет:

- макетом оболочки;
- макетами страниц;
- визуальным представлением;
- опциональным клиентским поведением через `template.js`.

---

3. Перспективные изменения

Раздел собирает возможности, которые ещё не входят в действующий формат: пока шаблон на них полагаться не может. При внедрении такая возможность переходит в основные разделы, а формат получает новую версию (см. «История версий»).

3.1 Расширенные интерактивы вопросов

Переопределение механики стандартных интерактивов средствами `template.js`.
Предполагаемая регистрация рендерера:

```
TestBuilder.template.registerInteractionRenderer("single", renderer);
TestBuilder.template.registerInteractionRenderer("multiple", renderer);
TestBuilder.template.registerInteractionRenderer("matching", renderer);
TestBuilder.template.registerInteractionRenderer("ranking", renderer);
```

Контракт рендерера:

```
{
  render(ctx),
  mount(root, ctx),
  getAnswer(root),
  setAnswer(root, answer),
  clearAnswer(root),
  setLocked(root, locked),
  showCorrectState(root, ctx),
  destroy(root)
}
```

Форматы ответов совпадают с теми, что принимает Core:

```
single: 2
multiple: [0, 3]
matching: { "0": 2, "1": 0 }
ranking: [2, 0, 1, 3]
```

3.2 Закрепление файловых версий шаблона

Выбранная версия шаблона (`templateVersion`) сохраняется для диагностики и миграций; хранение каждой прежней файловой версии не требуется. Возможное развитие — закрепление теста за точной файловой версией шаблона.

3.3 Система renderer-плагинов

Расширяемый реестр рендереров показателей результата помимо встроенных `core.*` (описан в §8.2.1.3 как перспективный).

4. Структура ZIP шаблона

Рекомендуемая структура:

```
template-id/
  manifest.json
  shell.html
  preview.png
  demo/
    course.json

  layouts/
    start.html
    question.html
    content.html
    results.html
    question-single.html
    system-locked.html
    report.html
    report.adaptive.html

  partials/
    topic-nav.html
    progress.html
    header.html
    footer-actions.html

  styles/
    base.css
    theme.css
    report.css

  scripts/
    template.js

  assets/
    fonts/
    images/
    icons/
```

`manifest.json` является источником истины. Дополнительные файлы разрешены, но все файлы, на которые есть ссылки, должны существовать внутри ZIP.

Внешние URL запрещены. Все CSS, JS, шрифты, изображения, иконки и другие ресурсы должны быть локальными файлами внутри SCORM ZIP.

Сгенерированный SCORM-пакет включает выбранный шаблон, а не полную библиотеку шаблонов. Кроме него в пакет попадают резервные макеты и стили стандартного шаблона — но только для системных экранов, которые выбранный шаблон не объявляет.

5. Манифест

5.1 Базовая структура

```
{
  "id": "corporate",
  "name": "Corporate",
  "version": "1.0.0",
  "templateApiVersion": "1.0",
  "description": "",

  "layouts": {
    "shell": "shell.html",
    "question": "layouts/question.html",
    "content": "layouts/content.html",
    "results": "layouts/results.html",
    "start": "layouts/start.html",
    "review": "layouts/review.html",
    "section-results": "layouts/section-results.html",
    "system.blocked": "layouts/system-blocked.html",
    "report": "layouts/report.html",
    "report.adaptive": "layouts/report.adaptive.html"
  },

  "contentTemplates": [
    {
      "key": "intro.hero",
      "label": "Введение: крупный заголовок и изображение",
      "kind": "intro",
      "pageKind": "content.intro",
      "layoutFile": "layouts/content-intro.html",
      "placeholders": [
        {
          "key": "title",
          "type": "text",
          "label": "Заголовок",
          "required": true,
          "maxLength": 120,
          "textFit": {
            "mode": "autoFitFont",
            "defaultFontSize": 36,
            "minFontSize": 24,
```

```
          "maxFontSize": 36,
          "allowAuthorFontSize": false,
          "overflow": "warn"
        }
      },
      {
        "key": "subtitle",
        "type": "text",
        "label": "Подзаголовок",
        "required": false,
        "maxLength": 500,
        "textFit": {
          "mode": "fixed",
          "defaultFontSize": 20,
          "allowAuthorFontSize": true,
          "allowedFontSizes": [16, 18, 20, 22],
          "overflow": "warn"
        }
      },
      {
        "key": "heroImage",
        "type": "image",
        "label": "Изображение",
        "required": false
      }
    ]
  },
  {
    "key": "info.textWithImage",
    "label": "Информация: текст и изображение",
    "kind": "info",
    "pageKind": "content.info",
    "layoutFile": "layouts/content-info.html",
    "placeholders": [
      { "key": "title", "type": "text", "label": "Заголовок", "required": true },
      { "key": "lead", "type": "text", "label": "Вводный текст", "required": false },
      { "key": "body", "type": "richText", "label": "Основной текст", "required": true },
      { "key": "image", "type": "image", "label": "Изображение", "required": false }
    ]
  },
  {
    "key": "summary.progressRing",
    "label": "Итог: кольцевая диаграмма прогресса",
```

```
      "kind": "summary",
      "pageKind": "content.summary",
      "layoutFile": "layouts/content-summary.html",
      "placeholders": [
        { "key": "title", "type": "text", "label": "Заголовок", "required": true },
        {
          "key": "progressChart",
          "type": "resultField",
          "label": "Показатель прогресса",
          "required": true,
          "allowedPaths": [
            "progress.active.percent",
            "progress.question.percent",
            "progress.page.percent",
            "result.scorePercent",
            "sectionResult.percent"
          ],
          "defaultPath": "progress.active.percent",
          "allowedRenderers": ["core.ringChart", "core.progressBar", "core.textMetric"],
          "defaultRenderer": "core.ringChart"
        }
      ]
    },
    {
      "key": "report.standard",
      "label": "Отчёт: стандартный",
      "kind": "report",
      "layoutFile": "layouts/report.html",
      "styleFile": "styles/report.css",
      "isDefault": true,
      "settings": [
        { "key": "headline", "type": "text", "label": "Заголовок отчёта", "default":
"Итоги" },
        { "key": "backgroundImage", "type": "image", "label": "Подложка страницы" }
      ]
    },
    {
      "key": "report.adaptive.standard",
      "label": "Отчёт: уровни",
      "kind": "report.adaptive",
      "layoutFile": "layouts/report.adaptive.html",
      "styleFile": "styles/report.css",
      "isDefault": true,
```

```
    "settings": []
  }
],

"rendererPlugins": [
  {
    "key": "core",
    "version": "1.0.0",
    "source": "core",
    "renderers": ["textMetric", "badge", "progressBar", "ringChart", "segmentedProgress"]
  }
],

"systemPages": [
  {
    "id": "system.locked",
    "layout": "layouts/system-locked.html",
    "purpose": "blocking",
    "countInProgress": false,
    "allowNavBack": false
  }
],

"partials": {
  "topicNav": "partials/topic-nav.html",
  "progress": "partials/progress.html",
  "header": "partials/header.html",
  "footerActions": "partials/footer-actions.html"
},

"assets": {
  "styles": ["styles/base.css", "styles/theme.css"],
  "scripts": ["scripts/template.js"],
  "fonts": ["assets/fonts/Inter.woff2"],
  "images": ["assets/images/logo.svg"],
  "preview": "preview.png"
},

"preview": {
  "demoData": "demo/course.json",
  "defaultRoute": "start",
  "routes": [
    "start",
```

```
      "content.intro",
      "content.info",
      "question.single",
      "question.multiple",
      "question.matching",
      "question.ranking",
      "content.summary",
      "results",
      "system.locked"
    ],
    "viewports": ["desktop", "mobile"]
  },

  "params": [
    {
      "key": "brand.primaryColor",
      "type": "color",
      "label": "Основной цвет",
      "default": "#0066cc",
      "group": "Бренд",
      "cssVar": "--tb-brand-primary",
      "validation": {
        "required": true
      }
    }
  ],

  "capabilities": {
    "navigation": ["linear", "free", "locked"],
    "sidebar": true,
    "progress": ["questions", "pages"],
    "timer": true,
    "contentPages": ["start", "intro", "info", "summary", "html"],
    "questionTypes": ["single", "multiple", "matching", "ranking"],
    "customInteractions": false,
    "runtimeApi": "1.0"
  }
}
```

5.2 Обязательные поля манифеста

Обязательные:

- `id`
- `name`
- `version`
- `templateApiVersion`
- `contentTemplates` (непустой массив, минимум один элемент — требует схема манифеста)
- `layouts.shell`
- `layouts.question`
- `layouts.content`
- `layouts.results`
- `assets`
- `assets.preview`
- `preview`
- `params`
- `capabilities`

Опциональные:

- `description`
- `layouts.start`
- детализированные макеты вопросов, например `question.single`
- детализированные макеты контентных страниц, например `content.info`
- `rendererPlugins`
- `partials`
- `systemPages`
- варианты отчёта (`contentTemplates[]` с `kind: "report"` / `report.adaptive` , §8.4) и парные им ключи `layouts.report` / `layouts["report.adaptive"]`
- варианты БЛОКОВ документа (`contentTemplates[]` с `kind: "report.block"` , §8.4.7)
- `reportDocument` — документ отчёта по умолчанию, по видам (§8.4.8)
- `labels[]` — надписи интерфейса итогов, доступные автору для переформулировки/выключения (§8.3.1)
- `resultsBlockOrder` — состав и порядок подблоков итогов по экранам (§8.3.3)

Отчёт объявлять не обязательно: шаблон без вариантов отчёта его не теряет — страница собирается макетом стандартного шаблона с оформлением активного (§8.4.4). Надписи и порядок подблоков объявлять тоже не обязательно: шаблон без `labels[]` печатает свои жёсткие строки макета, а без `resultsBlockOrder` получает зашитый порядок «Общий балл, По шкалам, По показателям, По темам» (§8.3.3).

5.3 Резервный выбор макетов

Core выбирает макет сначала по самому специфичному ключу, затем откатывается к общему ключу.

Примеры:

```
question.single   → question
question.multiple → question
content.intro     → content
content.info      → content
content.summary   → content
content.html      → content
start             → content
system.blocked    → system page layout → content
report            → layoutFile варианта → макет вида из стандартного шаблона
report.adaptive   → layoutFile варианта → макет вида из стандартного шаблона
```

У отчёта отката на общий макет НЕТ: осмысленного «макета отчёта по умолчанию» внутри шаблона не существует, поэтому вариант обязан объявить свой `layoutFile`, а единственный резерв — одноимённый вид стандартного шаблона (§8.4.4).

Обязательный минимум:

```
{
  "layouts": {
    "shell": "shell.html",
    "question": "layouts/question.html",
    "content": "layouts/content.html",
    "results": "layouts/results.html"
  }
}
```

5.4 Системные страницы

Шаблоны могут объявлять системные страницы для неучебных экранов: блокировки, предупреждения, промежуточные экраны, сертификаты или экраны после результата.

Поддерживаемые значения `purpose`:

```
blocking
warning
interstitial
postResult
certificate
custom
```

Системные страницы не влияют на оценку. `countInProgress` и `allowNavBack` определяют поведение прогресса и навигации.

> **Реализация.** Встроенный `default` объявляет системные экраны не массивом `systemPages[]`, а ключами в `layouts`: `system.blocked`, `system.transition` (адаптивный переход), а также узлы раздела `review`, `section-results`, `section-intro`. Полный массив `systemPages[]` с `purpose` / `countInProgress` / `allowNavBack` схемно не валидируется, хотя входит в контракт.

5.5 Контракт предпросмотра и демонстрационного набора данных

Предпросмотр является частью контракта шаблона, а не произвольной страницей шаблона. Он нужен для галереи, проверки загружаемых ZIP и ручной оценки поведения шаблона до активации.

Шаблон обязан объявить:

- статический предпросмотр в `assets.preview`;
- живой предпросмотр в `preview`;
- демонстрационный набор данных в `preview.demoData`.

`assets.preview` - локальный путь внутри ZIP к `png`, `jpg`, `jpeg`, `webp` или `svg`. Файл используется в галерее и списке шаблонов. Внешние URL запрещены.

`preview` описывает Core-owned live preview. Шаблон не должен полагаться на отдельный `preview.html` как на контрактный entrypoint: такой файл может существовать как вспомогательный стенд разработчика, но административный UI и smoke-проверка запускают шаблон через `shell.html`, манифест, layouts, ресурсы и демонстрационный набор.

```
type TemplatePreviewContract = {
  demoData: string;
  defaultRoute?: PreviewRoute;
  routes?: PreviewTarget[];
  viewports?: Array<"desktop" | "tablet" | "mobile">;
};

type PreviewTarget =
  | PreviewRoute
  | {
      route: PreviewRoute;
      label?: string;
      pageId?: string;
      templateKey?: string;
    };

type PreviewRoute =
  | "start"
  | "content.intro"
  | "content.info"
  | "content.summary"
  | "content.html"
  | "question.single"
  | "question.multiple"
  | "question.matching"
  | "question.ranking"
  | "review"
  | "section-results"
  | "results"
  | `system.${string}`;
```

Требования к `preview`:

- `demoData` указывает на локальный JSON-файл внутри ZIP;
- `defaultRoute` определяет первый экран живого предпросмотра, по умолчанию `start`;
- `routes` перечисляет экраны, которые UI и smoke-проверка должны открыть; если поле не задано, Core строит список из обязательных возможностей манифеста;
- если в пакете несколько `contentTemplates[]` одного `pageKind`, `routes[]` должен различать их через `templateKey` или `pageId`;
- `viewports` задаёт контрольные размеры предпросмотра; если поле не задано, Core проверяет `desktop` и `mobile`;

- все маршруты должны соответствовать объявленным layouts, `contentTemplates`, `systemPages` и `capabilities`;
- live preview работает без SCORM API, LMS, сетевых запросов и внешних ресурсов.

Демонстрационный набор данных - это стабильный JSON, совместимый с публичным runtime context. Он должен покрывать все маршруты из `preview.routes`, все заявленные типы вопросов, динамические `resultField` placeholders, progress, результаты и системные страницы.

Минимальный формат:

```
type PreviewDemoDataset = {
  schemaVersion: "1.0";
  locale?: string;
  params?: Record<string, unknown>;
  course: {
    title: string;
    mode?: "standard" | "adaptive";
    navigation?: "linear" | "free" | "locked";
    topics: PreviewTopic[];
    contentPages: PreviewContentPage[];
    questions: PreviewQuestion[];
  };
  runtime: {
    route: PreviewRoute;
    progress: {
      active: PreviewProgress;
      question: PreviewProgress;
      page: PreviewProgress;
    };
    result: PreviewResult;
    sectionResult?: PreviewResult;
    state?: Record<string, unknown>;
  };
};

type PreviewTopic = {
  id: string;
  title: string;
  status?: "locked" | "available" | "completed";
};

type PreviewContentPage = {
  id: string;
  type: "start" | "intro" | "info" | "summary" | "html";
  route?: PreviewRoute;
  templateKey?: string;
  topicId?: string;
  values: Record<string, unknown>;
};

type PreviewQuestion = {
  id: string;
```

```
  topicId?: string;
  type: "single" | "multiple" | "matching" | "ranking";
  prompt: string;
  options?: Array<{ id: string; text: string; correct?: boolean }>;
  pairs?: Array<{ id: string; left: string; right: string }>;
  order?: string[];
  feedback?: {
    text?: string;
    correctAnswerPublic?: Record<string, unknown>;
  };
};

type PreviewProgress = {
  current: number;
  total: number;
  percent: number;
};

type PreviewResult = {
  score: number;
  maxScore: number;
  scorePercent: number;
  status: "notStarted" | "inProgress" | "passed" | "failed" | "partial";
};
```

Требования к значениям:

- `schemaVersion` обязателен; несовместимая версия блокирует активацию шаблона;
- `params` переопределяет значения по умолчанию из `manifest.params` только для preview;
- для каждого `manifest.contentTemplates[]` должна быть хотя бы одна демонстрационная `contentPages[]` с тем же `templateKey`;
- `contentPages[].values` должен соответствовать `placeholders[]` выбранного `templateKey`;
- значения `resultField` должны использовать только `allowedPaths`, `allowedRenderers` и валидные `rendererOptions`;
- `questions[]` должен содержать по одному примеру для каждого типа из `capabilities.questionTypes`;
- `progress.*.percent`, `result.scorePercent` и `sectionResult.scorePercent` должны находиться в диапазоне `0..100`;
- демонстрационные данные не должны содержать персональные данные реальных пользователей, production ID, токены, LMS-поля или сетевые URL;
- Core может использовать один и тот же demo dataset для авторского preview и smoke-проверки.

При запуске live preview Core:

1. загружает `manifest.json`;
2. валидирует `assets.preview`, `preview` и `preview.demoData`;
3. строит публичный runtime context из `PreviewDemoDataset`;
4. рендерит `defaultRoute` через обычный runtime pipeline;
5. даёт UI возможность переключать маршруты из `preview.routes`;
6. логирует diagnostics с route, viewport, renderer key и ошибкой, если rendering/fallback был задействован.

Ошибки контракта preview делятся так:

- отсутствующий `assets.preview`, `preview` или `preview.demoData` - блокирующая ошибка;
- невалидный JSON demo dataset - блокирующая ошибка;
- route, для которого нет layout/template/capability, - блокирующая ошибка;
- отсутствие покрытия необязательного route - предупреждение;
- fallback renderer в preview - предупреждение, если страница осталась работоспособной;
- необработанная ошибка runtime в preview - блокирующая ошибка.

---

6. Параметры шаблона

Шаблоны определяют параметры через расширенный `params[]` с dot-keys, группами, значениями по умолчанию и валидацией.

Поддерживаемые типы:

```
color
text
number
boolean
select
image
font
asset
```

Пример:

```
{
  "key": "progress.mode",
  "type": "select",
  "label": "Прогресс",
  "options": ["questions", "pages", "hidden"],
  "default": "questions",
  "group": "Навигация"
}
```

Умолчание решает шаблон: `default` у параметров, которые читает ядро

Часть параметров ядро читает само, а не через CSS: схему уровней (`levelScheme` и цвета своей схемы), вид шкал и показателей, показ максимума шкалы, окраску полос подтем (`breakdownBarFill` , §10.6). Для них `default` манифеста действует ровно так же, как для `cssVar` и `dataAttr` ниже: если автор значение не трогал, ядро берёт умолчание ШАБЛОНА, а не своё. Так шаблон задаёт вид теста, в котором ничего не настраивали, а редактор показывает автору то значение, которое на самом деле применяется.

До 3.12.0 эти параметры брали жёсткое умолчание ядра, а `default` манифеста только показывался в редакторе. У всех поставляемых шаблонов оба умолчания совпадали, поэтому поведение прежних тестов не изменилось. Составные ключи (`progress.mode`) таким способом не дополняются.

Выбор картинкой: `optionPreviews`

Параметр `select` вправе объявить картинку на вариант — путь к СВОЕМУ файлу пакета:

```
{
  "key": "brandLogo",
  "type": "select",
  "label": "Логотип",
  "options": ["plain", "b2b"],
  "optionLabels": { "plain": "Без направления", "b2b": "B2B" },
  "optionPreviews": {
    "plain": "assets/logos/plain.png",
    "b2b": "assets/logos/b2b.png"
  }
}
```

Тогда редактор рисует не выпадающий список, а карточки с изображениями. Смысл в том, что выбирают ОБЛИК: словом «B2O» его не описать, и без картинки автор выбирал бы вслепую и шёл сверяться в предпросмотр.

Значением может быть путь строкой ЛИБО пара под темы интерфейса:

```
{
  "optionPreviews": {
    "b2b": { "light": "assets/logos/b2b-light.png", "dark": "assets/logos/b2b-dark.png" }
  }
}
```

Редактор берёт картинку по теме, в которой сидит АВТОР, а не по теме теста: лок, нарисованный под светлый фон, на тёмном редакторе нечитаем. Не хватает половины пары — берётся вторая.

Правила:

- пути — обычные ссылки манифеста: файл обязан быть в пакете (иначе импорт блокируется как `FILE_MISSING`), внешний `http(s)`-адрес запрещён, файл не считается неиспользованным;
- значение вне двух допустимых форм (не строка и не объект с `light` / `dark`) — блокировка: молча проигнорированное превью выглядит как пропавшая картинка;
- вариант без превью рисуется карточкой со значком-заглушкой — это нормальный случай для варианта со смыслом «ничего не показывать»;
- превью у варианта, которого нет в `options`, — предупреждение: картинка игнорируется, выбор работает;
- `optionPreviews` у типа, отличного от `select`, — предупреждение, поле не учитывается;
- вариант без картинки рисуется карточкой с пустой миниатюрой и подписью;
- поле необязательное: без него `select` остаётся списком, как и раньше.

Значение как атрибут сцены: `dataAttr`

`cssVar` кладёт значение параметра в кастомное свойство. Свойство — только ЗНАЧЕНИЕ: по нему нельзя выбрать правило, а шаблон, меняющий картинку по выбранному варианту, выбирает именно селектором. Для этого параметр объявляет `dataAttr`:

```
{
  "key": "brandLogo",
  "type": "select",
  "options": ["plain", "b2b"],
  "dataAttr": "data-brand-logo",
  "default": "plain"
}
```

Оба хоста кладут значение в этот атрибут на КОРЕНЬ сцены — там же, где живут CSS-переменные параметров и тема (`data-theme`, §6.1). В пакете это `<html>`, в вебе — host теневого дерева. CSS шаблона выбирает по нему:

```
[data-brand-logo="b2b"] .tb-scene__logo { background-image: url("…");
}
```

Правила:

- имя обязано быть вида `data-имя` (строчные латинские буквы, цифры, дефис) — иначе импорт блокируется (`PARAM_DATA_ATTR_INVALID`): хост произвольное имя не пишет, и шаблон молча рисовал бы запасной вариант;
- значение приводится к строке как есть; у числа единицы НЕ дописываются — это не размер, а ключ;
- нет значения у теста — берётся `default` манифеста; нет и его — атрибут не ставится;
- поле необязательное и обратно-совместимое.

Селектор, рассчитанный на оба хоста, пишется парой — в вебе класс/атрибут темы висит на host снаружи теневого дерева, и потомковый селектор до него не достаёт:

```
.ou--dark .tb-scene__logo,
:host(.ou--dark) .tb-scene__logo { … }
```

Семантика progress.mode

Параметр `progress.mode` управляет тем, что считается единицей прогресса в runtime.

Режим	Единица прогресса	Знаменатель
`questions`	Каждый вопрос, на который дан ответ	Общее число вопросов в тесте
`pages`	Каждый переход навигации (`Core.next()`)	Число navigable-экранов, фиксируется при загрузке
`hidden`	Не отображается	—

Правила подсчёта для `pages` :

- Знаменатель фиксируется при инициализации flow и не меняется в процессе прохождения.
- Каждый вызов `Core.next()` , приводящий к переходу на новый экран, увеличивает числитель на 1.
- Страницы `start` и `results` включены в подсчёт — они являются частью навигационного flow.
- Каждый вопрос считается отдельной единицей, а не весь блок вопросов темы: это обеспечивает плавный рост прогресса без скачков.
- Страница `system.blocked` исключена: её показ не изменяет счётчик прогресса. Blocked-состояние не является частью нормального flow.

- В режиме `pages` `progress.active.*` совпадает с `progress.page.*`; в режиме `questions` — с `progress.question.*`.

Core передаёт параметры в контекст макета, а также генерирует CSS-переменные (CSS custom properties) в браузере для параметров с `cssVar`.

Пример:

```
{
  "key": "brand.primaryColor",
  "type": "color",
  "default": "#0066cc",
  "cssVar": "--tb-brand-primary"
}
```

CSS, сгенерированный в браузере:

```
<style id="tb-template-vars">
  :root {
    --tb-brand-primary: #0066cc;
  }
</style>
```

6.1 Темы шаблона

Шаблон может поставлять несколько палитр. Стилями это выражается блоками `theme.css` (`:root`, `@media (prefers-color-scheme: dark)`, `:root[data-theme="dark"]`), но платформе этого НЕ достаточно: палитры объявляются в манифесте.

```
"themes": [
  { "id": "light", "label": "Светлая" },
  { "id": "dark",  "label": "Тёмная" }
]
```

Поле	Обязательность	Смысл
`themes`	нет	Палитры шаблона. Нет поля или меньше двух записей — платформа считает, что тем нет
`themes[].id`	да	Идентификатор из закрытого перечня платформы: `light`, `dark`
`themes[].label`	да	Подпись пункта переключателя и колонки в таблице цветов

Перечень закрыт так же, как реестр типов полей (§8.2.1): идентификатор вне перечня — ошибка шаблона, а не молча пропущенная запись. Проверка выполняется при загрузке И при активации; при загрузке дополнительно сверяется, что у каждой объявленной темы есть хотя бы один свой токен в стилях.

Что делает Core:

- **Выбор вида теста.** Автор фиксирует палитру (`light` / `dark`) или отдаёт её системной настройке участника (`auto`). Значение хранится у теста, а не у шаблона.
- **Значение цвета на каждую палитру.** Цветовые параметры хранятся отдельно по темам; нецветовые — одним значением, они от палитры не зависят.
- **Постановка темы на сцену.** Закреплённая палитра ставится атрибутом `data-theme` на корень сцены: `<html>` в SCORM-пакете, shadow-host на веб-хосте. При `auto` атрибут НЕ ставится — иначе медиа-запрос шаблона не получит своей очереди.
- **Печать переопределений.** Правила той же структуры, что и в `theme.css` , добавляются ПОСЛЕ стилей шаблона: специфичность равна, побеждает порядок.

На веб-хосте условие обязано идти функциональной формой — `:host([data-theme="dark"])` , `:host(:not([data-theme="light"]))` . Суффиксная форма `:host[data-theme="dark"]` невалидна, и браузер отбрасывает правило целиком: тема молча перестаёт применяться. По той же причине при переносе стилей шаблона в теневой корень условие переносится ВНУТРЬ скобок.

6.2 Миграция параметров при обновлении шаблона

Правило:

- тот же `key` и тот же `type` -> сохранить значение;
- старый ключ отсутствует -> игнорировать/удалить;
- новый ключ с `default` -> использовать `default` ;
- новый ключ без `default` -> автор должен заполнить значение.

UI должен показывать отчёт:

```
2 параметра сохранены, 1 параметр больше не используется, 1 новый параметр требует значения.
```

Будущее расширение: явные `paramMigrations` в манифесте для сложных enterprise-шаблонов.

---

7. Контракт оболочки

`shell.html` определяет внешний макет плеера.

Обязательные элементы оболочки:

```
<div data-slot="page"></div>

<button data-nav="next"></button>
<button data-action="answer-submit"></button>
<button data-action="test-finish"></button>
```

> **Реализация.** Структурный валидатор и браузерная проверка работоспособности требуют от оболочки только `data-slot="page"` (код ошибки `SHELL_CONTRACT`). Маркеры `data-nav` / `data-action` привязываются Core, когда присутствуют, но **не являются** обязательными для прохождения валидации/проверки: встроенный `default` их в `shell.html` не объявляет, а действия Core навешивает делегированием по `[data-action]` поверх экранов, отрисованных `renderScreenInto`. Жёсткая проверка привязки `next/ answer-submit/test-finish` относится к перспективным расширенным интерактивам (§3).

Опциональные элементы оболочки:

```
<button data-nav="prev"></button>
<div data-slot="progress"></div>
<div data-slot="topic-nav"></div>
<div data-slot="feedback"></div>
<div data-slot="timer"></div>
<div data-slot="breadcrumb"></div>
```

Оболочка может предоставлять резервные кнопки действий. Макеты страниц также могут объявлять локальные кнопки. Если для страницы существуют локальные действия, Core может скрыть или отключить резервные кнопки оболочки на этой странице.

7.1 Монтирование оболочки (`mountShell`)

По умолчанию Core монтирует экраны в собственный базовый контейнер, а `shell.html` служит только внешним каркасом. Флаг манифеста `"mountShell": true` включает ПРЯМОЕ монтирование оболочки: Core подставляет `shell.html` целиком и рендерит каждый экран внутрь её `#app` (`data-slot="page"`), заменяя базовый контейнер.

Флаг обязателен для шаблонов с ФИКС-СЦЕНОЙ — когда CSS шаблона привязан к структуре оболочки (например `.tb-frame > .tb-stage > #app`, сцена фиксированного размера или на container-query единицах `cqh` / `cqw`). Без `mountShell` такой шаблон рендерится вне своей сцены, и его оформление не применяется. То же монтирование выполняет и веб-хост в предпросмотре: оба хоста читают `mountShell` из манифеста, поэтому автор видит фикс-сцену одинаково в SCORM и в превью. Шаблон без фикс-сцены (как встроенный `default`) флаг не объявляет.

Пример локального действия:

```
<button data-nav="next" data-local-action="true">
  <span>{{ nav.nextLabel }}</span>
</button>
```

Core привязывает обработчики и обновляет состояние. Он не должен перезаписывать пользовательский HTML кнопки, если кнопка не пустая.

Core управляет состоянием кнопок навигации/действий через:

- `hidden`
- `disabled`
- `aria-disabled`
- `data-state`
- `data-visible`
- классы состояния, например `is-hidden` и `is-disabled`

---

8. Контракт макетов страниц

8.1 Макеты вопросов

Обязательные слоты:

```
<div data-slot="question-text"></div>
<div data-slot="question-interaction"></div>
```

> **Реализация.** Текст вопроса монтируется в `data-slot="question-text"` — это и есть имя слота, которое проверяет валидатор (код `QUESTION_CONTRACT`) и заполняет рендерер (`question-text` + `question-interaction`, см. `server/scorm/templates/default/layouts/question.html`). Имя `question-prompt` из ранних черновиков спецификации устарело; источник истины — `question-text`.

Опциональные слоты:

```
<div data-slot="question-media"></div>
<div data-slot="question-feedback"></div>
<div data-slot="question-hint"></div>
<div data-slot="question-counter"></div>
<div data-slot="question-topic"></div>
<div data-slot="question-difficulty"></div>
```

Core вставляет:

- формулировку вопроса в `question-text`;
- стандартный рендерер интерактива в `question-interaction`;
- медиа в `question-media` и обратную связь в `question-feedback`, если эти слоты присутствуют.

> **Реализация.** Из опциональных слотов рантайм реально заполняет только `question-media` и `question-feedback`. Слоты `question-hint` / `question-counter` / `question-topic` / `question-difficulty` управляемыми НЕ являются: подсказка приходит внутри `question-interaction` (элемент с классом `question-hint`), а счётчик выводится через `data-path` (`state.questionCounterLabel`).

Вопрос «Сценарий». Для этого типа вопроса ядро кладёт в `question-interaction` обложку сценария (размытый первый экран и «Пройти» либо итог прежнего прогона), а по «Пройти» открывает окно правил и полноэкранный плеер. Разметку и стили даёт ядро (встроенные `style` и токены `--ou-*`), поэтому шаблон ничего не объявляет и не стилизует: ни слотов, ни классов, ни параметров. То же касается карточки пункта-сценария на странице-маршрутизаторе: это `router-topic-card` с модификатором `router-topic-card--scenario`, оформляется как карточка темы.

Картинки на карточках хаба. Карточки маршрутизатора строит ядро (`shared/flow/router-hub`), шаблон их только оформляет. Картинка карточки — свойство ОФОРМЛЕНИЯ страницы-маршрутизатора, а не пункта теста: вариант вида `router` объявляет настройку `cardImages` типа `itemImages` (§8.2.1), и только через такой вариант ядро кладёт картинки на карточки. Шаблон, которому нужны обе раскладки, объявляет два варианта маршрутизатора — без картинок и с картинками; поставляемые шаблоны называют второй `router.menu-images` («Меню карточек с картинками»). Карточка с картинкой несёт модификатор `router-topic-card--with-img` и дочерний элемент `<span class="router-topic-card__img"><img src="…" alt=""></span>` сразу после `router-topic-card__top`; где стоит картинка и какого она размера, решает CSS шаблона. Карточка без картинки модификатора и элемента не получает. Картинка декоративная (`alt=""`): название карточки уже говорит, что это за пункт. Экспорты `TBTemplate`, которыми пользуется пакет (`renderSimCover`, `renderSimRulesDialog`, `simCoverState` и др.), — внутренний слой ядра, а не часть формата.

8.2 Макеты и шаблоны контентных страниц

Контентные страницы работают по модели PowerPoint slide layouts:

- **тип страницы** определяет смысл страницы в flow: `intro`, `info`, `summary`, `html`, `blocked`;
- **content template** в SCORM-шаблоне определяет скелет данных страницы: placeholders, их типы, обязательность, ограничения и layout;
- **экземпляр страницы в тесте** хранит значения placeholders, которые заполнил автор;
- **layout** отображает эти значения, но не является источником содержимого.

Это означает, что автор не заливает произвольную вёрстку в шаблонную страницу. Автор выбирает подходящий скелет страницы и заполняет его структурированные поля. Свободный HTML остаётся отдельным типом `content.html` для технических сценариев и не является основным способом создания страниц.

Поддерживаемые режимы определения страницы:

Mode	Назначение	Когда использовать
`template`	Страница создаётся по `manifest.contentTemplates[]`; автор заполняет placeholders	Основной режим для стандартизированных корпоративных страниц
`standard`	Страница использует каноническую схему Core для `intro/info/summary` и fallback layout	Для переносимых страниц, не завязанных на конкретный шаблон
`html`	Страница содержит санитизированный HTML и отображается через `content.html`	Только как escape hatch для технических авторов

Поддерживаемые kinds контентных страниц:

Откуда экран берёт макет — зависит от его класса (см. классы ниже таблицы). Системные экраны резолвятся по фиксированным ключам `layouts[]`; прочие контентные страницы — по `layoutFile` варианта, иначе по общему `layouts["content"]`. Когда шаблон не объявляет нужный системный экран, применяется fallback на стандартный шаблон с предупреждением в UI — колонка «Fallback» ниже.

> Отдельного вида страницы «галерея» больше нет: галерея — обычный вариант информационной страницы (`kind: "info"`) со своим `layoutFile`, и отдельного правила резолвинга для неё не существует. Индикатор навигации по подряд идущим страницам макет рисует из вычисленного контекста `page.*` (§10), а не из значений, введённых автором. Свойства страницы (подпись кнопки `nextLabel`, фон `backgroundImage`, идентификатор последовательности `sequenceId`) объявляются отдельным блоком `settings[]` варианта — по природе они не содержимое. Когда вариант, к которому привязана страница, в выбранном шаблоне отсутствует, страница рендерится вариантом по умолчанию своего `kind` (см. §8.2.2), а `templateKey` в базе не переписывается: замену подтверждает автор

> через диалог сопоставления. Оба поставляемых шаблона несут сетку из шести раскладок контентной страницы в двух семействах (обычная и галерейная) — двенадцать вариантов; их перечень — эталонный, но не нормативный.

Резолвинг макета делится на три класса (по фактическому рантайму `server/scorm/template/app/render/`).

- **Системные экраны** (`results`, `question`, `review`, `section-results`, «Введение раздела», `system.blocked`) резолвятся по ФИКСИРОВАННОМУ ключу `layouts[<key>]`. Вариант в `contentTemplates[]` для них не задаёт макет — он нужен лишь для привязки страницы и для определения fallback (объявлен ли `kind`). В частности «Введение раздела» рендерится по ключу `layouts["section-intro"]`, а НЕ по `layoutFile` варианта `intro`; если этого ключа нет, экран падает в общий контентный рендер.
- **Стартовый экран** (`start`) резолвится по `layoutFile` ВЫБРАННОГО варианта старта, иначе по фиксированному `layouts["start"]`. Шаблон может объявить несколько `contentTemplates[]` с `kind: "start"`, каждый со своим `layoutFile` (напр. `start.image-right` — колонка-иллюстрация справа); базовый помечается `isDefault: true`. Автор выбирает раскладку через «Сменить вариант» на строке «Старт», и ОБА хоста рендерят выбранный макет (SCORM-рантайм `startPage.resolveStartLayout`, веб-хост `GET /screen-template/start`). Отсутствие выбранного варианта или его файла — fallback на `layouts["start"]` (и далее на стандартный шаблон). Контекст строит общий `buildStartState` независимо от раскладки; иллюстрацию несёт СВОЙСТВО СТРАНИЦЫ `image` того варианта, который его объявил (`settings[]`), — вариант без свойства иллюстрации не получает. Значение попадает в контекст как `design.startImageUrl` через общий `startImageForVariant`. Тем же путём вариант объявляет тумблеры плиток обложки (§10.13).
- **Прочие контентные страницы** (`info` / `summary` / `router` / `html`) резолвятся по `layoutFile` варианта, иначе по общему `layouts["content"]`. Галерея — частный случай `info` и отдельного правила не имеет.

Page kind	Назначение	Откуда макет	Fallback на стандартный шаблон
`start`	Стартовый экран теста (лендинг) — тест-уровневый, всегда	`layoutFile` выбранного варианта старта, иначе `layouts["start"]`	Да
`question`	Экран вопроса	`layouts["question"]`	Да
`results`	«Итоги теста» — итоговый результат всего теста, тест-уровневый, всегда	`layouts["results"]`	Да
`review`	«Обзор раздела/теста» перед завершением — системный узел	`layouts["review"]`	Да
`section-results`	Вычисляемые «Итоги раздела» — системный узел	`layouts["section-results"]`	Да
`content.intro`	«Введение раздела» (`before_topic`) — по одной на тему	`layouts["section-intro"]` , иначе общий контентный рендер	Да
`system.blocked`	Системная страница блокировки	`systemPages[].layout` / `layouts["system.blocked"]`	Да (файловый)
`content.info`	Информационная/ учебная страница; сюда же относится слайд галереи — вариант со своим `layoutFile`	`layoutFile` варианта, иначе `layouts["content"]`	Нет: недоступный вариант подменяется вариантом по умолчанию `kind` , замену подтверждает автор (§8.2.2)
`content.summary`	«Итог раздела» (`after_topic`) — по одной на тему; показывает результат РАЗДЕЛА	`layoutFile` варианта, иначе `layouts["content"]`	Нет: вариант по умолчанию `kind` (§8.2.2)
`content.router`	Меню тем (`router_by_topics`)	`layoutFile` варианта, иначе `layouts["content"]`	Нет: вариант по умолчанию `kind` (§8.2.2)
`content.html`	Санитизированный HTML-блок	`layouts["content"]`	Нет: общий контентный рендер

Fallback на стандартный шаблон применяется только к экранам с ВЫДЕЛЕННЫМ макетом (`start` , `results` , `question` , `intro` , `review` , `section-results` , `system.blocked`). Экраны, которые рендерятся общим `layouts["content"]` (`info` / `summary` / `router` / `html`), подменять нечем — их «резервом» служит сам общий контентный макет.

Отчёт о результатах (`report` , `report.adaptive`) в таблице выше не значится намеренно: это не экран прохождения, а документ, который обучающийся скачивает файлом. Объявляется он тем же `contentTemplates[]` , но живёт по своим правилам — свой резолвинг макета, своя среда стилей и свои проверки. Полный контракт — в §8.4.

`start` и `results` — тест-уровневые системные экраны (по одному на тест, в любом режиме). `review` и `section-results` — системные узлы раздела: «Обзор» перед завершением и вычисляемые «Итоги раздела» после него; они рендерятся своими рантайм-фазами и исключены из потока контентных страниц. `content.intro` — «Введение раздела» перед его вопросами (по одной на тему, только в режимах по темам). `content.summary` — устаревшая per-topic закладка «Итог раздела»: остаётся валидной для обратной совместимости, но её роль выполняет вычисляемый `section-results` .

8.2.1 `contentTemplates[]`

`manifest.contentTemplates[]` объявляет скелеты страниц, доступные автору при выбранном шаблоне. Один SCORM-шаблонный пакет может содержать несколько content templates. Это штатный механизм для вариантов страниц: например несколько `content.info` , несколько `content.summary` , разные варианты стартового экрана (`start`) или итогов теста (`results`). Стандартный (`default`) шаблон обязан объявить хотя бы по одному варианту каждого системного `kind` : `start` , `results` , `router` , `questions` , `intro` , `review` , `section-results` , а также по одному варианту каждого вида отчёта — `report` и `report.adaptive` (он — системный fallback; `review` и `section-results` — узлы раздела; `summary` остаётся валидным `kind` для обратной совместимости, но в обязательный набор уже не входит).

Множественность работает так:

- `contentTemplates[]` - массив, а не одиночный объект;
- `contentTemplates[].key` уникален внутри `manifest.contentTemplates[]` ;
- каждый `contentTemplates[]` элемент обязан иметь человекочитаемое `label` ;
- несколько templates могут иметь одинаковый `pageKind` , если у них разные `key` ;
- несколько templates могут использовать один и тот же layout, если их placeholders совместимы с этим layout;
- авторский UI при добавлении страницы показывает все доступные templates текущего пакета и может группировать их по `pageKind` ;
- авторский UI показывает автору `label` , а не технический `key` ;
- `templateKey` экземпляра страницы всегда указывает на конкретный `contentTemplates[].key` , а не только на `pageKind` или layout.

Минимальная структура:

```
{
  "key": "info.textWithImage",
  "label": "Информация: текст и изображение",
  "kind": "info",
  "pageKind": "content.info",
  "layoutFile": "layouts/content-info.html",
  "description": "Страница с заголовком, основным текстом и иллюстрацией",
  "placeholders": [
    {
      "key": "title",
      "type": "text",
      "label": "Заголовок",
      "required": true,
      "maxLength": 120,
      "textFit": {
        "mode": "fixed",
        "defaultFontSize": 32,
        "allowAuthorFontSize": false,
        "overflow": "error"
      }
    },
    {
      "key": "body",
      "type": "richText",
      "label": "Основной текст",
      "required": true,
      "textFit": {
        "mode": "autoFitFont",
        "defaultFontSize": 20,
        "minFontSize": 14,
        "maxFontSize": 20,
        "allowAuthorFontSize": false
      },
    },
    {
      "key": "image",
      "type": "image",
      "label": "Изображение",
      "required": false,
      "constraints": {
        "aspectRatio": "16:9"
      }
```

```
    }
  ]
}
```

> **Реализация.** Схема манифеста (`contentTemplateEntrySchema`) требует у элемента `key` , `label` и `kind` (виды из §8.2); `pageKind` и `placeholders[]` опциональны. Путь к макету указывается полем `layoutFile` — относительным путём к файлу, напр. `"layouts/content-info.html"` . Ключа `layout` в контракте нет. `layoutFile` читается рантаймом только для НЕсистемных контентных страниц (`info` / `summary` / `router` / `html`); системные экраны и «Введение раздела» резолвятся по фиксированным ключам `layouts[]` (см. таблицу выше). См. эталонный `server/scorm/templates/default/manifest.json` .

Типы placeholders — ЗАКРЫТЫЙ перечень. Placeholder описывает СОДЕРЖИМОЕ, вставляемое в макет:

```
text
textarea
richText
html
image
resultField
```

> Перечень приведён к фактически поддерживаемому и объявлен однократно в общем коде (`shared/template/field-types.ts`): он питает и контрол редактирования, и правило отрисовки значения, а привязка «тип — контрол» задаётся полным отображением, поэтому пропуск контрола становится ошибкой сборки, а не молчаливой деградацией поля в однострочный ввод. Из прежнего списка `number` , `boolean` и `select` переезжают в `settings[]` (см. ниже) — по природе это настройки страницы, а не её содержимое; `video` , `file` и `actionLabel` не поддерживаются и из контракта исключены. Неизвестный тип — ошибка валидации (§17.1).

`settings[]` — свойства страницы

Кроме содержимого вариант может объявить СВОЙСТВА страницы — то, что управляет её поведением и оформлением, но не вставляется в макет как контент. Раздел необязателен: его отсутствие означает, что у страниц этого варианта настраиваемых свойств нет.

```
{
  "key": "gallery.card",
  "label": "Галерея: карточка",
  "kind": "info",
  "layoutFile": "layouts/gallery.html",
  "placeholders": [
    { "key": "header", "type": "text", "label": "Заголовок" },
    { "key": "cardText", "type": "richText", "label": "Текст карточки" }
  ],
  "settings": [
    { "key": "sequenceId", "type": "sequence", "label": "Идентификатор последовательности"
},
    { "key": "nextLabel", "type": "text", "label": "Подпись кнопки", "default": "Далее" },
    { "key": "backgroundImage", "type": "image", "label": "Фон страницы" }
  ]
}
```

Типы настроек — закрытый перечень: `number`, `boolean`, `select`, `text`, `image`, `sequence`, `scaleAppearance`, `itemImages`.

Правила:

- настройка может объявлять значение по умолчанию (`default`) и обязательность (`required`); значение по умолчанию подставляется при создании страницы, а незаполненная обязательная настройка блокирует сохранение так же, как незаполненный обязательный placeholder;
- редактор структуры не имеет собственных полей: форма страницы состоит ИСКЛЮЧИТЕЛЬНО из объявленных `placeholders[]` и `settings[]`;
- `sequence` — идентификатор последовательности страниц. Подряд идущие страницы одного участка структуры с одинаковым непустым значением образуют последовательность; ядро само вычисляет число точек индикатора и текущую позицию и передаёт их макету через `page.*` (§10). Вариант, не объявивший `sequence`, поля не показывает и точек не получает;
- `itemImages` — картинки по пунктам теста: карта «ключ пункта → картинка», где ключ — `topic:<id>` темы или `scenario:<id>` пункта-сценария, а значение — `{ "url": "…", "name": "…" }` (имя файла нужно только редактору). Редактор рисует строку на каждый пункт в порядке хаба, со своей загрузкой. Сейчас тип читает одна настройка — `cardImages` варианта вида `router` (§8.1); картинки пунктов, ушедших из теста, ядро снимает при сохранении теста и при сборке пакета;
- значения настроек хранятся отдельно от содержимого страницы и вместе с ним попадают в SCORM-пакет и в снапшот публикации;
- добавление раздела обратно совместимо: `templateApiVersion` не повышается.

Вариант может пометить себя `isDefault: true` — это делает его вариантом по умолчанию для своего `kind` (см. §8.2.2). Если пометки нет ни на одном варианте `kind`, по умолчанию считается ПЕРВЫЙ объявленный вариант этого `kind`.

8.2.2 Недоступный вариант: подмена вариантом по умолчанию

Автор привязывает страницу к варианту (`templateKey`). Сменив шаблон оформления теста, автор может получить страницы, чей вариант новый шаблон не объявляет. Такая страница не ломается и не исчезает: она рендерится ВАРИАНТОМ ПО УМОЛЧАНИЮ своего `kind` (см. выше). Правило едино для всех трёх мест, где страница превращается в экран, — SCORM-рантайм, веб-хост и предпросмотр — и живёт в общем коде (`shared/template/content-page.ts → resolveContentTemplate`).

Подмена — решение ОТРИСОВКИ, а не правки данных:

- `templateKey` в базе не переписывается. Поэтому привязка переживает возврат прежнего шаблона, а «Структура» продолжает помечать страницу как требующую сопоставления;
- окончательную замену подтверждает автор через диалог «Сменить вариант», который показывает теряемые при переходе значения. До подтверждения — только подмена при отрисовке;
- если шаблон не объявляет НИ ОДНОГО варианта нужного `kind`, подменять нечем — страница деградирует до простого контентного макета (заголовок и текст), как и раньше;
- список тестов помечает жёлтым знаком тест, в котором есть хотя бы одна страница с недоступным вариантом (число — в подсказке, по клику — переход в «Структуру»). Признак считается на сервере за один запрос на всю видимую страницу списка, а не по запросу на тест.

8.2.1.1 Политика размера текста

Для placeholders типов `text`, `textarea` и `richText` шаблон должен явно определить поведение текста при переполнении блока. Модель следует логике PowerPoint text box:

> **Реализация.** Подгонку размера текста (`autoFitFont`, `growBox`, `overflow`) реально применяет только SCORM-рантайм (`applyTextFit` в `templateCore.js`, покрыто тестами). Веб-хост (единый рендерер `renderScreenInto`) `textFit` пока НЕ применяет — там это только метаданные плейсхолдера. Значение `allowAuthorFontSize`/ручной размер сохраняются на сервере независимо от хоста.

`textFit.mode`	Поведение	Когда использовать
`fixed`	Размер шрифта и размер блока фиксированы; переполнение диагностируется/обрезается согласно `overflow`	Для строгих брендовых слайдов
`autoFitFont`	Core уменьшает размер шрифта в пределах `minFontSize` / `maxFontSize`, чтобы текст поместился	Для заголовков и коротких блоков
`growBox`	Размер шрифта фиксирован, а блок увеличивается по вертикали под текст в рамках layout constraints	Для длинных информационных блоков

Пример:

```
{
  "key": "title",
  "type": "text",
  "label": "Заголовок",
  "required": true,
  "maxLength": 120,
  "textFit": {
    "mode": "autoFitFont",
    "defaultFontSize": 36,
    "minFontSize": 24,
    "maxFontSize": 36,
    "allowAuthorFontSize": false,
    "overflow": "warn"
  }
}
```

Поля `textFit`:

Поле	Назначение
`mode`	`fixed`, `autoFitFont`, `growBox`
`defaultFontSize`	Размер шрифта по умолчанию из шаблона
`minFontSize`	Минимальный размер для `autoFitFont`
`maxFontSize`	Максимальный размер для `autoFitFont` и ручного ввода
`allowAuthorFontSize`	Разрешает автору вручную менять размер шрифта для этого placeholder
`allowedFontSizes`	Допустимые значения ручного размера, если нужен список вместо диапазона
`overflow`	`warn`, `clip`, `scroll`, `error`
`maxHeight`	Максимальная высота блока для `growBox`, если layout допускает ограничение

Требования к `textFit`:

- если `allowAuthorFontSize = false`, UI не показывает управление размером шрифта для этого placeholder;
- если `allowAuthorFontSize = true`, UI ограничивает размер шрифта диапазоном `minFontSize` / `maxFontSize` или отдельным `allowedFontSizes`;
- ручной размер шрифта сохраняется как override конкретной страницы, а не как изменение layout шаблона;
- `autoFitFont` выполняется Core/runtime, а не произвольным кодом шаблона;
- при `fixed` и переполнении Core должен применить `overflow` и записать диагностику;
- при `growBox` Core может увеличивать вертикальный размер блока только в пределах ограничений layout, чтобы не ломать соседние элементы.

Ручной размер шрифта хранится отдельно от текстового значения:

```
{
  "templateKey": "intro.hero",
  "values": {
    "title": "Перед началом раздела",
    "subtitle": "Короткое описание"
  },
  "placeholderStyles": {
    "subtitle": {
      "fontSize": 18
    }
  }
}
```

`placeholderStyles[placeholderKey].fontSize` валиден только если соответствующий placeholder имеет `allowAuthorFontSize = true`. Для `richText` ручной размер применяется к контейнеру placeholder; произвольные inline font-size внутри richText не поддерживаются.

Общие требования к `contentTemplates[]`:

- `contentTemplates[].key` уникален внутри `manifest.contentTemplates[]`;
- `contentTemplates[].label` обязателен, не пустой, человекочитаемый и пригоден для отображения в UI выбора страницы, preview и диагностике;
- `contentTemplates[].label` не должен быть техническим идентификатором, path или повторением `key`; если нужно пояснение, используется опциональный `description`;
- `placeholders[].key` уникален внутри конкретного `contentTemplate`;
- `placeholders[].label` обязателен для каждого поля, которое показывается автору в форме;
- `type`, `required`, `maxLength`, `options`, `constraints` используются UI для формы заполнения;
- для `text` / `textarea` / `richText` задана `textFit`-политика; если не задана, Core использует `fixed` и пишет предупреждение в валидации шаблона;
- `richText` хранится как структурированный документ или ограниченный HTML, прошедший санитизацию;
- `image/video/file` ссылаются на локальные assets, которые упаковываются в SCORM ZIP;
- layout не должен ожидать значения, не объявленные в `placeholders`.

Объём допустимого HTML в значениях автора

Разметку принимают только типы `richText` и `html`; значения прочих типов экранируются при отрисовке. Санитизация СЕРВЕРНАЯ и выполняется дважды: при сохранении страницы (автор получает список удалённого) и повторно при сборке SCORM-пакета; рендерер вставляет очищенное значение без дополнительной обработки.

Модель — чёрный список. Удаляются: `<script>`, `<iframe>`, `<svg>`, `<object>`, `<embed>`, `<link>`, `<meta>` (вместе с содержимым), все атрибуты-обработчики `on*`, `javascript:` в `href` / `src` (значение заменяется на `#`) и внешний `http(s)://` в `src` — на любом теге, а также в `href` у всех тегов, КРОМЕ `<a>` и `<area>`. Разделение по смыслу: `src` называет ресурс, который страница грузит сама, и внешний адрес ломает автономность пакета, тогда как `href` ссылки — направление, по которому учащийся идёт сам, и до клика не запрашивается ничего. Внешняя ссылка поэтому СОХРАНЯЕТСЯ и получает `target="_blank" rel="noopener noreferrer"` (атрибуты, заданные автором, не перекрываются): внутри LMS пакет живёт в кадре, и переход в том же кадре увёл бы учащегося с незавершённой попытки. Остальная разметка проходит: форматирование, структура, таблицы, контейнеры, атрибуты `class` / `id` / `style`, внутренние ссылки и изображения из `/uploads/`.

Отсюда обязанность шаблона: ссылка в авторском тексте должна ЧИТАТЬСЯ как ссылка. Браузерное умолчание не годится — оно не знает палитры шаблона, а у `<a>` без адреса его нет вовсе. Поставляемые шаблоны объявляют правило для областей авторской разметки (обложка,

текст страницы, заметки раздела, текст вопроса, вводный блок итогов, обратная связь) в `theme.css` и одно общее `.tb-report a` в `report.css`.

Ограничения, которые обязан учитывать шаблон: тег вне списка запрещённых пройдёт (включая `<style>`, `<form>`, `<input>`), `data:`-URI не блокируются, разбор ведётся регулярными выражениями без DOM. Поэтому макет и его CSS рассчитаны на чужую разметку: оформление изолируется классами компонентов и не опирается на глобальные селекторы.

> Объём допустимого HTML не меняется — он равен описанному выше; дополнительные перечни разрешённых или запрещённых тегов не вводятся. Меняется ввод: у текстового поля появляется переключатель режимов (простой текст / форматированный текст / HTML), где ТИП поля задаёт потолок (`html` — все три режима, `richText` — первые два, `text` / `textarea` — только простой). В режиме форматированного текста работает единая для всех шаблонов панель (полужирный, курсив, зачёркнутый, списки, ссылка, очистка), а вставка из внешнего источника нормализуется; в режиме HTML запрещённая конструкция НЕ сохраняется — выдаётся ошибка со списком найденного. Объявления `allowedMarks` / `allowedBlocks` из контракта исключены. Ресурсы шаблона (изображения, стили) адресуются относительным путём (`images/logo.png`); базовый путь подставляет ядро на каждом хосте, внешний `http(s)`-адрес РЕСУРСА (`src`) остаётся запрещённым — в отличие от адреса ссылки, который сохраняется (см. выше).

8.2.1.2 Динамические placeholders и визуализация показателей

Динамический контент страницы отображается через placeholder типа `resultField`. Шаблон объявляет, какие runtime-пути и какие контролируемые renderers допустимы, а автор выбирает источник данных и подпись. Шаблон не вычисляет результат и не получает произвольный JavaScript для диаграмм.

Пример декларации:

```
{
  "key": "progressChart",
  "type": "resultField",
  "label": "Показатель прогресса",
  "required": true,
  "allowedPaths": [
    "progress.active.percent",
    "progress.question.percent",
    "progress.page.percent",
    "result.scorePercent",
    "sectionResult.percent"
  ],
  "defaultPath": "progress.active.percent",
  "allowedRenderers": ["ringChart", "progressBar", "segmentedProgress", "textMetric"],
  "defaultRenderer": "ringChart",
  "format": {
    "type": "percent",
    "decimals": 0
  }
}
```

Экземпляр страницы хранит выбор автора:

```
{
  "templateKey": "summary.progressRing",
  "values": {
    "title": "Ваш прогресс",
    "progressChart": {
      "path": "progress.active.percent",
      "renderer": "ringChart",
      "label": "Пройдено"
    }
  }
}
```

Layout размещает только placeholder:

```
<h1>{{ page.values.title }}</h1>
<div data-placeholder="progressChart"></div>
```

Core выполняет безопасный pipeline:

1. валидирует `path` по `allowedPaths`;
2. читает значение из публичного runtime context;
3. нормализует значение по `format`;
4. валидирует `renderer` по `allowedRenderers` и registry renderer plugins;
5. вызывает renderer через контролируемый runtime API;
6. вставляет результат в `data-placeholder`.

Стандартные runtime-пути:

Path	Назначение
`progress.active.current` / `progress.active.total` / `progress.active.percent`	Активный прогресс согласно `progress.mode`
`progress.question.current` / `progress.question.total` / `progress.question.percent`	Прогресс только по вопросам
`progress.page.current` / `progress.page.total` / `progress.page.percent`	Прогресс по всем страницам flow
`result.scoreRaw` / `result.scoreMax` / `result.scorePercent`	Итоговый результат теста
`result.status`	Итоговый статус, например `passed` / `failed`
`sectionResult.scoreRaw` / `sectionResult.scoreMax` / `sectionResult.percent`	Результат текущего раздела для `content.summary`
`sectionResult.status`	Статус текущего раздела
`result.{name}`	Пользовательские показатели результата
`retake.availableDate`	Дата доступного повторного прохождения для `system.blocked`
`retake.effectiveToday`	Нормализованное «сегодня», от которого считаются производные величины; равно `todayDate`, если часы не пришлось нормализовать

Стандартные renderers поставляются как plugin `core` и используются по полным ключам `core.textMetric`, `core.ringChart` и т.д.

Renderer	Назначение	Рекомендация UI/UX
`core.textMetric`	Крупное число/значение с подписью	Для точного результата, статуса, даты
`core.badge`	Компактный статус	Для `passed` / `failed`, уровня, зоны риска
`core.progressBar`	Линейная полоса прогресса	Лучший default для процесса прохождения
`core.ringChart`	Кольцевая диаграмма одного процента	Для одного hero-показателя, не использовать пачками
`core.segmentedProgress`	Полоса из сегментов по количеству шагов/вопросов	Для небольшого числа вопросов или разделов
`core.questionTiles`	Строка/сетка кубиков: каждый кубик = вопрос	Для диагностического прогресса, когда `total ≤ 40`; при большем числе нужна агрегация
`core.sectionList`	Список разделов со статусами/процентами	Для тестов с явными разделами
`core.scaleBars`	Несколько горизонтальных шкал	Для компетенций и многошкальных результатов

> **Реализация.** Реально зарегистрированы пять рендереров: `core.textMetric`, `core.badge`, `core.progressBar`, `core.ringChart`, `core.segmentedProgress` (`shared/template/renderers.ts`). `core.questionTiles`, `core.sectionList`, `core.scaleBars` в действующий набор не входят (перспективные, §3). Реестр устойчив к ошибкам: рендерер не из `allowedRenderers`, неизвестный или упавший откатывается к `core.textMetric`; `path` не из `allowedPaths` даёт пустое значение.

Рекомендации:

- для обычного текущего прогресса использовать `progressBar`;
- для стартовой/итоговой страницы с одним главным числом использовать `ringChart` + текстовый процент;
- для тестов до 40 вопросов можно использовать `questionTiles`; для больших тестов использовать `segmentedProgress` с группировкой или `progressBar`;
- цвет не должен быть единственным носителем смысла: renderer обязан выводить текст/label или доступный `aria-label`;
- thresholds цветов задаются шаблоном/rendererOptions, а не произвольной логикой автора;
- динамические renderers не влияют на результат, SCORM-статусы и навигацию.

8.2.1.3 Renderer plugins

Renderer plugin - расширение, которое добавляет один или несколько контролируемых renderers для `resultField`. Он отвечает только за визуальное представление уже рассчитанных данных. Plugin не может менять ответы, результат, SCORM-статусы, навигацию, `TEST_DATA` или runtime state.

> Полноценная система плагинов ниже относится к перспективным изменениям (§3) и пока не действует: доступны только встроенные `core.*` (см. §8.2.1.2). SCORM-рантайм содержит рудиментарный загрузчик `manifest.rendererPlugins` (инжект `<script>` по `plugin.path` / `src`) и хук регистрации `TestBuilder.renderers.register(id, fn)`, где рендерер — чистая функция `(value, options) ⇒ string`, а не `mount()`-инстанс из контракта ниже. Реестр/ `source` (`core` / `template` / `registry`), проверка `version`, валидация `optionsSchema` и тип `RendererRegistry` / `DynamicRenderer` НЕ реализованы. Веб-хост (единый рендерер) загрузки плагинов не имеет вовсе — там доступны только пять `core.*`. Эталон объявляет `"rendererPlugins": []`.

Источники renderer plugins:

Source	Назначение
`core`	Встроенные renderers платформы, доступны всегда
`template`	Renderer plugin, поставляемый внутри конкретного SCORM-шаблона
`registry`	Администрируемый plugin из общего registry, копируется в SCORM ZIP при экспорте

Манифест шаблона или экспортированного пакета объявляет используемые plugins:

```
{
  "rendererPlugins": [
    {
      "key": "core",
      "version": "1.0.0",
      "source": "core",
      "renderers": ["textMetric", "badge", "progressBar", "ringChart"]
    },
    {
      "key": "rtk.progress",
      "version": "1.2.0",
      "source": "template",
      "entry": "renderers/progress/index.js",
      "styles": ["renderers/progress/style.css"],
      "renderers": [
        {
          "key": "questionTiles",
          "label": "Кубики вопросов",
          "valueTypes": ["progress"],
          "optionsSchema": {
            "type": "object",
            "properties": {
              "maxTiles": { "type": "number", "default": 40 },
              "shape": { "type": "string", "enum": ["square", "rounded"] }
            }
          }
        }
      ]
    }
  ]
}
```

Полный ключ renderer строится как `{pluginKey}.{rendererKey}`. Например:

```
{
  "allowedRenderers": ["core.progressBar", "core.ringChart", "rtk.progress.questionTiles"],
  "defaultRenderer": "core.progressBar"
}
```

Runtime API plugin:

```
type DynamicRendererPlugin = {
  key: string;
  version: string;
  register(registry: RendererRegistry): void;
};

type RendererRegistry = {
  register(renderer: DynamicRenderer): void;
};

type DynamicRenderer = {
  key: string;
  label: string;
  valueTypes: Array<"number" | "percent" | "string" | "boolean" | "date" | "progress" |
"list">;
  optionsSchema?: JsonSchema;
  mount(root: HTMLElement, input: RendererInput): RendererInstance | void;
};

type RendererInput = {
  value: unknown;
  rawValue: unknown;
  path: string;
  label?: string;
  format?: Record<string, unknown>;
  options?: Record<string, unknown>;
  context: PublicRuntimeContext;
  theme: RuntimeTheme;
};

type RendererInstance = {
  update?(input: RendererInput): void;
  destroy?(): void;
};
```

Ограничения безопасности и совместимости:

- renderer plugin загружается только из SCORM ZIP, внешние URL запрещены без отдельной политики;
- `eval`, `Function`, inline script injection и запись в глобальные объекты Core запрещены;
- plugin не имеет доступа к SCORM API напрямую;
- plugin получает только публичный runtime context и не должен читать приватные данные;

- plugin обязан обрабатывать пустое/ `null` значение и показывать fallback;
- ошибка plugin не должна ломать страницу: Core заменяет renderer на fallback `core.textMetric` или показывает диагностику placeholder;
- `options` автора валидируются по `optionsSchema` до сохранения и перед runtime;
- renderer обязан поддерживать доступность: текстовая подпись, `aria-label`, отсутствие зависимости только от цвета;
- если plugin не найден или версия несовместима, страница остаётся открываемой с fallback renderer и диагностикой.

UI автора для `resultField` должен:

1. показать только renderers, перечисленные в `allowedRenderers` и доступные в registry;
2. после выбора renderer построить форму `rendererOptions` по `optionsSchema`;
3. показывать preview на демонстрационных данных;
4. сохранять выбор в значении placeholder:

```
{
  "path": "progress.question.percent",
  "renderer": "rtk.progress.questionTiles",
  "label": "Вопросы",
  "rendererOptions": {
    "maxTiles": 30,
    "shape": "square"
  }
}
```

8.2.2 Как автор определяет страницу

Основной сценарий:

1. Автор нажимает **“Добавить страницу”**.
2. Выбирает место в flow.
3. Выбирает режим:
   - **по шаблону** (`template`);
   - **стандартная страница** (`standard`);
   - **HTML-страница** (`html`).
4. Для режима `template` выбирает один из `manifest.contentTemplates[]` текущего SCORM-шаблона.
5. UI строит форму по `placeholders[]`.
6. Автор заполняет значения placeholders.
7. Предпросмотр показывает страницу через layout выбранного шаблона.
8. При сохранении в тест записываются `templateKey`, `pageKind`, `values`, позиция и служебные настройки.

Пример экземпляра страницы:

```
{
  "id": "page-1",
  "mode": "template",
  "templateKey": "info.textWithImage",
  "pageKind": "content.info",
  "topicId": "topic-1",
  "position": "before_topic",
  "values": {
    "title": "Перед началом раздела",
    "body": {
      "format": "richText",
      "document": {}
    },
    "image": {
      "assetId": "asset-123",
      "src": "assets/content/page-1.png"
    }
  },
  "autoAdvance": false
}
```

Runtime page получает значения в `page.values`:

```
{
  "id": "page-1",
  "kind": "content.info",
  "layoutKey": "content.info",
  "templateKey": "info.textWithImage",
  "values": {
    "title": "Перед началом раздела",
    "body": {},
    "image": {
      "src": "assets/content/page-1.png"
    }
  }
}
```

Layout использует значения как обычные path-only переменные:

```
<h1>{{ page.values.title }}</h1>
<div data-placeholder="body"></div>
<img data-placeholder="image">
```

Core заполняет `data-placeholder` согласно типу placeholder. Для `richText` Core вставляет только санитизированный результат. Для `image` Core выставляет локальный `src` и alt-текст.

Поведение необязательных placeholders (required: false):

- Если placeholder не заполнен автором, Core **не вставляет содержимое** в соответствующий `data-placeholder` и при необходимости скрывает обёртку через атрибут `data-placeholder-hide` (шаблон обязан поддерживать этот атрибут для всех `required: false` блоков).
- Шаблон не должен рассчитывать на наличие значения в необязательном placeholder — верстка должна корректно отображаться при пустом блоке без видимых артефактов (пустые рамки, зазоры).
- Валидация и сохранение формы блокируются только при незаполненных **обязательных** полях (`required: true`). Незаполненные необязательные поля не блокируют сохранение.
- UI формы редактирования страницы показывает под необязательными полями hint: “Необязательно — если не заполнено, блок не отображается.”

8.2.3 Совместимость при смене шаблона

Если страница создана в режиме `standard`, смена шаблона сохраняет содержимое и меняет только отображение.

Если страница создана в режиме `template`, она привязана к `templateKey` и схеме placeholders. При смене шаблона Core должен:

1. найти в новом шаблоне `contentTemplates[].key` с тем же `templateKey`;
2. если найден, применить новый layout к сохранённым values;
3. если не найден, показать автору состояние **“требуется сопоставление шаблона страницы”**;
4. позволить выбрать новый `contentTemplate` и вручную сопоставить совместимые placeholders;
5. не удалять старые values автоматически.

8.2.4 Стандартные и HTML-страницы

Режим `standard` использует канонические placeholders Core для переносимых страниц:

```
title
subtitle
media
body
primaryActionLabel
summaryFields
```

Режим `html` предназначен только для технических сценариев. HTML проходит санитизацию и отображается через `content.html` или `layouts.content`. Использование `html` должно быть явно видно в UI, потому что такая страница слабее стандартизирована, чем `template` / `standard`.

`content.intro` и `content.summary` — симметричные «закладки» раздела: `content.intro` рендерится перед вопросами раздела (`before_topic`), `content.summary` — после расчёта результата раздела (`after_topic`). Для `content.summary` Core дополнительно добавляет в публичный контекст готовый результат темы/раздела (`result.*` = результат РАЗДЕЛА, не всего теста — итог теста показывает макет `results`, §8.3). Шаблон может показать эти значения через placeholders типа `resultField`, но не рассчитывает их самостоятельно.

8.3 Макеты результатов

Макеты результатов могут использовать экранированные переменные и опциональные контролируемые слоты.

Опциональные слоты:

```
<div data-slot="results-summary"></div>
<div data-slot="result-variables"></div>
```

Пользовательские показатели результата остаются отдельной продуктовой функцией, но после вычисления публикуют значения в пространстве имён `result.*`.

Экран итогов несёт два уровня заголовков: зонтик «Ваш результат» и, под ним, до четырёх подблоков — сводка баллов, шкалы, показатели, темы, каждый со своим заголовком. Оба уровня надписаны надписями шаблона (§8.3.1), а состав и порядок подблоков — отдельной настройкой шаблона (§8.3.3). Та же иерархия и те же ключи действуют на адаптивных итогах, на итогах раздела и в отчёте.

Строка темы внутри подблока `topics` может нести ещё один уровень — строки разреза результата (подытоги по ключам оси, §10.6). Это необязательный блок внутри карточки: макет, который его не печатает, остаётся рабочим.

8.3.1 Надписи интерфейса: `labels[]`

Надпись — именованная строка интерфейса, которую объявляет ШАБЛОН с текстом по умолчанию, а автор теста может переформулировать или выключить (PRD-49). Заголовки блоков экрана итогов — «По шкалам», «Ваш результат» — до этого расширения были зашиты в макет: изменить формулировку или убрать заголовок, не убирая блок целиком, было нечем.

Надписи объявляются разделом ВЕРХНЕГО УРОВНЯ манифеста `labels[]`, а не полем внутри `contentTemplates[].settings[]`: одна надпись действует сразу на нескольких экранах (итоги, адаптивные итоги, итоги раздела, отчёт), тогда как `settings[]` по природе принадлежит одному варианту.

```
{
  "key": "recommendations.courses",
  "group": "Группы рекомендаций",
  "label": "Подпись группы курсов",
  "default": "Пройти обучение",
  "defaults": { "report": "Рекомендации по курсам" }
}
```

Поле	Обязательность	Смысл
`key`	да	Семантический ключ; по нему макет обращается к надписи (`labels.results.scales`) и по нему же строится путь дерева контекста (§10.5)
`group`	да	Группировка полей в редакторе — пятнадцать надписей сплошным списком нечитаемы
`label`	да	Подпись поля для автора
`default`	да	Текст по умолчанию, общий для всех экранов
`defaults.<экран>`	нет	Умолчание конкретного экрана, когда оно отличается от общего

Экраны, для которых надпись может объявить собственное умолчание: `results`, `results.adaptive`, `section-results`, `report`. Приём «общее умолчание плюс умолчание экрана» здесь тот же, каким `resultsBlockOrder` (§8.3.3) объявляет состав и порядок подблоков — шаблон следует одной конвенции в обоих разделах.

Раздел необязателен: шаблон, не объявивший ни одной надписи, печатает свои жёсткие строки макета — подраздел «Заголовки и подписи» такому шаблону в редакторе не показывается.

Статические проверки при загрузке шаблона (`validateLabelDeclarations`, код `LABELS_INVALID`, см. §17.1):

- `key` обязателен и уникален внутри `labels[]`;
- объявленная надпись обязана иметь `default`;
- ключи не могут быть взаимными префиксами (`results` вместе с `results.heading`) — контекст резолвит путь как ДЕРЕВО (§10.5), а не как плоскую карту, и префиксный ключ не может одновременно быть строкой-листом и веткой с вложенными ключами.

8.3.2 Хранение значения и три состояния надписи

Значения автора хранятся ОТДЕЛЬНО от объявления: манифест несёт умолчания, тест — только отклонения от них.

- Общий словарь — `tests.design_settings_json.labels`: ключ отсутствует — печатается текст шаблона.
- Порядок подблоков — `tests.design_settings_json.resultsBlockOrder` (§8.3.3).
- Переопределения ОТЧЁТА — `tests.report_settings_json.labels`, той же формы. Отчёт может говорить иначе, чем экран итогов, если автор этого захотел; пусто — как на экране итогов.

Новых колонок не заводится: оба словаря лежат в уже существующих `jsonb`-полях теста.

Значение одной надписи — ЗАПИСЬ с двумя необязательными полями, а не голая строка:

```
{ "on": false }
{ "on": true, "text": "Профиль стилей" }
```

Запись, а не строка, потому что у надписи ТРИ различимых состояния, и строкой их не развести: пустая строка неотличима от «поле никогда не открывали».

Состояние	Хранение	Что печатается
Не трогал	ключа в словаре нет	текст шаблона (`default` / `defaults.<экран>`)
Переформулировал	непустой `text` (`on` не `false`)	своя формулировка
Выключил	`{ "on": false }`	ничего — заголовок гасится, сам блок остаётся

Выключение — тумблер, а не очистка поля: у записи есть отдельный флаг `on`, поэтому «не трогал» и «выключил» различимы в данных, а не только по соглашению об пустой строке.

Разрешение идёт слоями поверх умолчания шаблона: словарь теста, затем (только для отчёта) переопределение отчёта. Слой либо ничего не говорит о ключе (пропуск — резолвер идёт глубже), либо гасит текст (`on: false` → пустая строка), либо задаёт свой текст. Одна функция на все хосты — `resolveLabels` (`shared/template/labels.ts`) — веб-хост, пакет SCORM и отчёт читают её ответ, ни один не считает надписи сам.

Перечень надписей ОТЧЁТА считается по макетам, а не объявляется. Редактор предлагает автору в настройках отчёта не все объявленные надписи, а только те, которые макеты вариантов отчёта действительно печатают — И ОБОЛОЧКИ, И БЛОКОВ ДОКУМЕНТА (§8.4.7). С разбором отчёта на блоки надписи переехали в них: зонтичный заголовок итогов живёт теперь в макете блока сводки, а не в оболочке, и сканер, читающий одну оболочку, предлагал бы автору половину словаря. Список собирается сканированием этих макетов на прямые пути `labels.<ключ>` (`readReportLabelKeys`, поле `reportLabelKeys` ответа `GET /api/templates/:id`) и отдаётся в порядке манифеста. Второй список в манифесте разошёлся бы с макетами молча, и автор продолжал бы включать заголовок, которого в документе нет. Для отчёта расчёт точен, а не приблизителен: состав и порядок подблоков (§8.3.3) документу не передаются, поэтому прямой путь `labels.<ключ>` — единственный способ попасть заголовку в PDF. Отсюда правило для разработчика шаблона: надпись, которую макет отчёта не печатает прямым путём, автору для отчёта не предложат. Шаблон, не объявивший ни одного варианта отчёта, берёт перечень «Стандартного» — тем же правилом, каким берёт его макеты (§8.4.4). Перечень считается на запрос, а не при регистрации шаблона: иначе он отставал бы от файлов шаблона до перерегистрации, как сам манифест.

8.3.3 Порядок и состав подблоков: `resultsBlockOrder`

Раздел манифеста верхнего уровня объявляет по каждому экрану, какие из пяти подблоков итогов (`summary`, `scales`, `indicators`, `topics`, `breakdown`) печатаются и в каком порядке.

```
"resultsBlockOrder": {
  "default": ["summary", "scales", "indicators", "topics", "breakdown"],
  "results.adaptive": ["topics", "scales", "indicators", "breakdown"]
}
```

Форма объявления — та же «общее умолчание плюс умолчание экрана», что и у надписей (§8.3.1): `default` отвечает за экраны, для которых нет отдельной записи, именованный ключ экрана (`results.adaptive`) переопределяет список целиком. Плоский массив вместо объекта — тоже валидная форма и равносилен единственному `default`: так шаблон, который экраны между собой не различает, объявляет один список на все.

Список экрана несёт СРАЗУ ДВА решения — состав и порядок. Ключа нет в списке экрана — на этом экране такого подблока не бывает: адаптивные итоги никогда не печатали сводку баллов, поэтому в их списке `summary` нет и не должно появляться. Порядок при этом читается позицией в массиве.

Раздел необязателен. Шаблон, не объявивший его, получает зашитое умолчание (`summary`, `scales`, `indicators`, `topics`, `breakdown`) — тот порядок, что макет печатал до появления этой настройки, плюс сводный разрез последним, — поэтому вид уже собранных тестов не меняется сверх переезда самих заголовков на зонтик и подзаголовки. Место `breakdown` в конце списка не

декоративное: шаг 3 разрешения дописывает в конец всё, чего сохранённый порядок автора не упоминает, поэтому тест, собранный до появления подблока, всё равно получит его именно там — объявить его в константе иначе значило бы разойтись с тем, что печатает живой тест.

Настройка автора теста — ОДНА на все экраны, а не по экрану (§8.3.2): список шаблона одного экрана определяет, какие из выбранных автором ключей на этом экране применимы, и что делать с ключом, которого шаблон не знает. Разрешение (`resolveBlockOrder` , `shared/template/results-order.ts`):

1. для рендерящегося экрана берётся список шаблона — умолчание экрана либо общее (`templateBlockOrder`);
2. сохранённый порядок автора фильтруется по этому списку: ключ, которого список экрана не содержит, отбрасывается — экран не может напечатать то, для чего шаблон не выделил место;
3. ключ, который список экрана знает, а сохранённый порядок ещё не упоминает (шаблон обновился и добавил подблок позже, чем автор в последний раз сохранял порядок), дописывается в конец в порядке шаблона — новый подблок не выпадает и не ломает то, что автор уже расставил.

Ключ вне закрытого перечня (`summary` / `scales` / `indicators` / `topics` / `breakdown`) отбрасывается на любом слое без ошибки: раздел — данные, а не код, и обязан принимать значение, не роняя рендер.

8.4 Макеты отчёта о результатах

Отчёт — PDF-файл с итогами попытки, который обучающийся скачивает действием `download-report` (флаг `result.nav.showReport` на экране итогов, `state.canDownloadReport` на стартовом экране в кулдауне). Это не снимок экрана результатов, а самостоятельный ДОКУМЕНТ со своей вёрсткой, и вёрстка эта принадлежит ШАБЛОНУ, а не ядру.

С версии 2.0.0 документ собирается из БЛОКОВ: вид `report` (и `report.adaptive`) объявляет ОБОЛОЧКУ — корень документа, — а разделы объявляются отдельными вариантами вида `report.block` (§8.4.7). Порядок и состав блоков в конкретном тесте задаёт автор теста; шаблон объявляет документ по умолчанию (§8.4.8). Шаблон, блоков не объявивший, печатает свою цельную раскладку ровно как раньше (§8.4.9).

Страницу рисует тот же браузерный рендерер, что и ученические экраны; конвейер экспорта растеризует её и укладывает в A4. Ядро отвечает за данные, за сборку документа и за файл, шаблон — за облик оболочки и каждого блока. Данные, из которых документ собирается, — публичный контекст отчёта (§10.2).

8.4.1 Виды и объявление варианта

Видов два, и они РАЗНЫЕ, а не варианты одного: у них разное содержание, и вариант одного режима не может быть выбран для другого.

`kind`	Режим теста	Что печатает
`report`	обычный	баллы, проценты, вердикт «пройден / не пройден»
`report.adaptive`	адаптивный	подтверждённые уровни по темам, без баллов

Вид определяется РЕЖИМОМ теста; подмены вида выбором автора нет.

Вариант объявляется записью `manifest.contentTemplates[]`:

- `layoutFile` — ОБЯЗАТЕЛЕН. Общего макета «на весь вид» у отчёта нет (§5.3);
- `styleFile` — необязательная таблица стилей страницы (§8.4.3);
- `isDefault` — ровно один вариант на каждый объявленный вид;
- `settings[]` — поля, которые автор теста заполняет в настройках теста;
- `placeholders[]` — НЕ применяются У ОБОЛОЧКИ: она не несёт содержимого, она его вмещает. Объявленный непустой `placeholders[]` — ошибка валидации (§17.1). Блоку `page` они, наоборот, разрешены: его текст пишет автор теста (§8.4.7).

Типы `settings[]` — общий закрытый перечень (§8.2.1) за вычетом `sequence`: последовательностей страниц у отчёта нет, поэтому такой тип отклоняется валидацией. Шаблон может объявить любое число вариантов каждого вида — например «с подложкой» и «строгий, под печать».

С версии 2.0.0 `layoutFile` вида `report` — это макет ОБОЛОЧКИ, а не всей страницы. Оболочка обязана быть ОДНИМ корневым узлом и не содержать разделов документа: движок вкладывает блоки её ПРЯМЫМИ детьми, и всякий контейнер между корнем и блоком ломает постраничную раскладку — она измеряет именно прямых детей корня (§8.4.6).

Типичная оболочка — корень с классом состояния и полями подложки и логотипа:

```
<div class="tb-report {{ report.verdictClass }}" style="{{#if
report.values.backgroundImage}}background-image: url({{ report.values.backgroundImage }});
{{/if}}">
{{#if report.values.logoImage}}<div class="tb-report__brand"><img src="{{
report.values.logoImage }}" alt=""></div>{{/if}}
</div>
```

Назначение поля: `scope`

Поле варианта отчёта может объявить, к чему оно относится:

`scope`	Где автор его правит	Что это за поля
`content`	«Настройки» → блок обратной связи	что попадёт в документ: диаграмма по шкалам, её предел, состав блоков
`appearance` (умолчание)	«Оформление» → «Отчёт»	как документ выглядит: подложка, логотип, типографские параметры

Признак нужен потому, что автор ищет эти поля в разных местах: содержание документа — рядом с обратной связью, которую получит слушатель, облик — рядом с шаблоном и брендингом. Разложить поля по двум экранам может только тот, кто их объявил.

Умолчание — `appearance`, и оно не нейтральное, а совместимое: до появления признака все поля показывались в «Оформлении», и шаблон, который о признаке не знает, оставляет их ровно там, где автор их находил. Непонятное значение трактуется как `appearance` и шаблон не отклоняет.

8.4.1.1 Картинки отчёта — файлы шаблона

Подложка, логотип и любая декоративная графика отчёта принадлежат ШАБЛОНУ: он кладёт файлы в свой пакет, перечисляет их в `manifest.assets.images` и объявляет полем `settings[]` типа `image`, чей `default` — путь внутри шаблона. Ядро не знает ни имён этих полей, ни их файлов.

```
{
  "key": "backgroundImage",
  "type": "image",
  "label": "Подложка страницы",
  "default": "assets/report/bg.png"
}
```

Значение поля может быть двух видов, и разрешаются они по-разному:

Значение	Что это	Как разрешается
`assets/report/bg.png`	файл шаблона (умолчание манифеста либо путь, введённый автором)	против базы ХОСТА: `template/…` внутри пакета, `/api/templates/<id>/assets/…` на вебе
`{ "url": "/uploads/media/x.png" }` или абсолютный URL	картинка автора теста	берётся как есть

Перед растеризацией хост инлайнит эти значения в data-URL: растеризатор снимает то, что уже лежит в документе, и ничего не догружает — незагруженная подложка молча меняет PDF. Картинка, которую прочитать не удалось, становится пустым значением, а не ошибкой: макет

гейтит свои строки на значении и печатает страницу без неё.

Отсюда правило для макета: он читает `report.values.<key>`, а не какое-то отдельное поле контекста. Ядро не поставляет отчёту ни подложки, ни логотипа по умолчанию — их даёт шаблон.

8.4.2 Как выбор автора доходит до выдачи

Автор теста выбирает вариант и заполняет его поля; выбор — свойство ТЕСТА, а не шаблона и не темы (§16). Разрешение выбора против манифеста — ОДНА функция на все хосты (`shared/report/report-variants.ts → resolveReportBake`): иначе отчёт в LMS расходился бы с тем, что автор видел в предпросмотре.

Хост	Когда разрешается	Где лежит результат
SCORM-пакет	при сборке (манифест виден только сборщику)	`TEST_DATA.designSettings.report` (`layoutKey`, `styleFile`, `values`); CSS варианта вложен в `styles.css`
Веб	при запросе результата попытки	поле `reportRender` ответа `/api/attempts/:id/result` (макет, CSS, переменные темы, значения полей)
Предпросмотр в настройках теста	в браузере автора	из бандла шаблона, на несохранённом черновике

Значения полей — правки автора поверх `default` манифеста; поле, которого выбранный вариант не объявляет, отбрасывается, поэтому смена варианта не тащит чужие значения. При смене варианта переживают переход только ключи, объявленные обоими вариантами с ОДНИМ типом.

8.4.3 Среда стилей отчёта

Отчёт рендерится в служебном контейнере ГЛАВНОГО документа, вне сцены. Контракт задан явно, потому что среды хостов расходятся ровно в одном месте: дизайн-система у обоих лежит в главном документе, а CSS ШАБЛОНА в SCORM-пакете лежит в главном документе, тогда как на вебе внедряется внутрь Shadow DOM экрана. Макет отчёта, опирающийся на `theme.css` или на слой сцены, выглядел бы верно в LMS и сломался бы в браузере.

Макету отчёта доступны:

1. **Компоненты дизайн-системы** (`.ou-*`) — как на любом экране.
2. **Токены темы и брендинг теста.** Хост записывает их CSS-переменными на КОНТЕЙНЕР отчёта (те же `buildTemplateCssVars` / `buildTemplateThemeCss`, что питают сцену, но с корневым селектором `.tb-report`). Работают `var(--ou-…)`, `hsl(var(--primary))`, `var(--font-sans)` и пинованная тема. Это ЕДИНСТВЕННЫЙ канал темы: `theme.css` целиком не подключается.
3. **Собственный `styleFile` варианта** — внедряется одинаково на обоих хостах.

Запрещены и отклоняются валидацией (§17.2):

- классы слоя сцены (`tb-scene*`): сцена описывает экран фиксированного вьюпорта, а отчёт — колонка шириной 595 px, печатаемая на A4;
- селекторы `:root`, `html`, `body` в `styleFile`: отчёт документом не является.

Запрет сделан исполнимым, а не пожеланием: корневой элемент макета обязан нести класс `tb-report`, а все селекторы `styleFile` — быть вложены в `.tb-report`.

Шрифты снимаются растеризатором из вычисленных стилей, поэтому шрифт, который должен попасть в PDF, обязан быть доступен в момент рендеринга (в пакете — из его ассетов, в вебе — из глобальных шрифтов приложения). Отсутствующий шрифт даёт не ошибку, а молча другой PDF — поэтому рендер отчёта входит в smoke-проверку (§17.3).

8.4.4 Деградация: шаблон без вида отчёта

Шаблон, не объявивший нужного вида, отчёта НЕ лишается. Страница собирается макетом одноимённого вида СТАНДАРТНОГО шаблона, а оформление (параметры брендинга, палитра, логотип) и значения полей берутся из АКТИВНОГО шаблона и настроек теста. Вместе с макетом из стандартного шаблона приезжает и его `styleFile` — иначе страница собралась бы без оформления.

Правило то же, что для системных экранов (§8.2), и работает на обоих хостах: в пакете — через резервные макеты, вложенные сборщиком, на вебе — повторным чтением из каталога `default`.

8.4.5 Кликабельные ссылки и вложение в пакет

Растровая страница сама по себе ссылок не несёт. Конвейер экспорта накладывает поверх снимка НАСТОЯЩИЕ ссылки PDF из двух источников:

- элемент макета с классом `pdf-link-btn` и атрибутом `data-url` — по координатам элемента. Без этого контракта рекомендованный курс в отчёте останется картинкой;
- ссылка `<a href>` с адресом `http(s)://` или `mailto:` — как правило, из авторского текста (страницы документа, вводный блок, рекомендации). Область строится ПО СТРОКАМ (`getClientRects`), поэтому перенесённая ссылка нажимается на каждой своей строке и не захватывает соседний текст. Якорь `#…` и относительный путь пропускаются: в PDF им вести некуда.

Пакет несёт каталог шаблона целиком, поэтому макет и картинки варианта в нём уже есть — под `template/` (а при деградации, §8.4.4, — под `template-default/`). Сверх этого сборщик вкладывает CSS выбранного варианта в общий `styles.css` и запекает разрешённые пути картинок: к моменту растеризации читать манифест и файлы из рантайма поздно. Пакеты, собранные до введения контракта, запечённого выбора не несут и собирают отчёт по каноническому виду — прежнее поведение.

На вебе картинки шаблона отдаются существующим роутом файлов шаблона (`GET /api/templates/:id/assets/*`), поэтому область сессии по ссылке-приглашению включает этот роут: без него ученик, пришедший по ссылке, получил бы отчёт без оформления.

8.4.6 Постраничная раскладка и принудительный разрыв

Отчёт — не одна длинная картинка: конвейер экспорта режет отрисованную страницу на листы A4. Разрезы раскладка ставит САМА — по высоте листа, по границам карточек и по нижним границам строк текста, — поэтому строка никогда не делится пополам. Решение это машинное: раскладка знает, что помещается на лист, но не знает, что документ читается разделами.

Приказать ей начать новый лист макет может пустым узлом с атрибутом `data-page-break`:

```
<section class="tb-report__card">…результаты по темам…</section>
<div data-page-break></div>
<section class="tb-report__card">…показатели…</section>
```

Контракт метки:

- **Атрибут, а не класс.** `data-page-break` стоит в одном ряду с `data-path` и `data-action`: это контракт РЕНДЕРЕРА, а не имя из стилевого пространства шаблона. Класс принадлежит шаблону, и внешний шаблон, переименовавший его у себя, потерял бы правило молча.
- **Стилей не требует.** Узел пуст и потому нулевой высоты, а разрез идёт по его ВЕРХНЕЙ границе: даже если шаблон случайно даст метке высоту, на бумаге она следа не оставит.
- **Любая глубина.** Метка действует прямым ребёнком корня отчёта, внутри карточки и внутри повтора `{{#each}}` — «каждое толкование показателя с новой страницы» пишется одной меткой в теле повтора. Внутри `{{#if}}` она печатается по тому же условию, что и остальная разметка, в том числе по значению поля варианта (§8.4.1), если разработчик шаблона отдаёт выбор автору.

Правила раскладки:

- принудительный разрыв СИЛЬНЕЕ всех прочих правил: и «карточку не рвём», и правило висячей строки, и предпочтение крупных границ отступают перед ним. Метка внутри карточки рвёт карточку вместе с её фоном и скруглениями — это осознанный выбор верстальщика;
- метка, над которой на текущем листе ничего нет (две подряд, метка в начале документа, метка сразу после другого разрыва), ИГНОРИРУЕТСЯ, как и метка после последнего содержимого: пустых листов механика не порождает;
- между двумя метками содержимое по-прежнему делится по высоте листа — раздел длиннее страницы режется обычным правилом, по нижним границам строк;
- **лист остаётся полноразмерным.** Разрыв меняет только то, где кончается СОДЕРЖИМОЕ; лист остаётся A4 с подложкой во всю высоту и ширину, полустраниц без фона не возникает.

Правило живёт в двух чистых модулях — `shared/report/paginate.ts` (сам разрез) и `shared/report/paginate-dom.ts` (измерение верхних границ меток в браузере), — которые зовут и конвейер PDF, и предпросмотр отчёта в редакторе. Поэтому скачанный файл, предпросмотр и отчёт внутри SCORM-пакета получают разрывы разом.

8.4.7 Блоки документа: вид `report.block`

Раздел документа объявляется записью `contentTemplates[]` вида `report.block`:

Поле	Обязательность	Смысл
`block`	обязательно	Ключ блока из закрытого перечня ниже
`kinds[]`	необязательно	Виды отчёта, которым служит вариант; без него — оба
`layoutFile`	обязательно	Макет раздела
`isDefault`	ровно один на пару «блок + вид»	Умолчание блока
`placeholders[]`	только у блока `page`	Поля авторской страницы
`settings[]`	необязательно	Свойства раздела

Перечень блоков ЗАКРЫТ продуктом: это то, что документ умеет рассказать о попытке. Шаблон вправе решать, КАК блок выглядит, но не вправе придумать одиннадцатый вид данных.

Ключ	Что печатает
`header`	Название документа, вердикт, слушатель, дата
`intro`	Вводный текст автора теста
`summary`	Сводка баллов
`topics`	Результаты по темам
`breakdown`	Сводный разрез по тесту
`scales`	Шкалы
`indicators`	Показатели
`recommendations`	Рекомендации
`courses`	Курсы
`events`	Мероприятия
`page`	АВТОРСКАЯ страница: заголовок и текст, которые пишет автор теста
`page-break`	Разрыв листа. Раскладки не имеет: это инструкция документу, а не раздел

Блок `page` — единственный, которому разрешён `placeholders[]`: его содержимое пишет автор теста, и типы полей те же, что у контентных страниц (§8.2.1). У оболочки `placeholders[]` по-прежнему запрещён (§8.4.1): оболочка не несёт содержимого, она его вмещает.

Шаблон вправе объявить несколько вариантов одного блока — «темы карточками» и «темы таблицей», «страница в одну колонку» и «в три». Автор теста выбирает вариант строкой документа; вариант, которого шаблон больше не объявляет, деградирует к умолчанию СВОЕГО блока, а не роняет блок целиком.

`kinds[]` нужен там, где блок печатается по-разному у обычного и адаптивного отчёта: карточка темы говорит то долей верных ответов, то подтверждённым уровнем. Вариант без `kinds[]` служит обоим видам — так объявляются блоки, у которых различий нет.

Блок `page-break` шаблоном НЕ объявляется: раскладки у него нет, печатает его сам движок узлом `data-page-break` (§8.4.6).

8.4.8 Документ по умолчанию: `reportDocument`

Корневой ключ манифеста объявляет, из чего документ состоит, пока автор теста его не трогал:

```
"reportDocument": {
  "report": ["header", "intro", "page-break", "topics", "summary", "recommendations"],
  "report.adaptive": ["header", "intro", "topics", "recommendations"]
}
```

Правила:

- список — ключи блоков в порядке печати;
- блок с данными в списке НЕ повторяется: второй «Сводный разрез» напечатал бы одни и те же числа дважды;
- `page-break` повторяется свободно: документ из трёх листов ставит два разрыва;
- вид, для которого состав не объявлен, печатается цельной раскладкой (§8.4.9) — даже если соседний вид на блоки уже переведён;
- блок, объявленный в списке, но не имеющий ни одного варианта, — ошибка валидации (§17.1).

Что происходит со СТАРЫМ тестом при смене шаблона, шаблона не касается: строки теста не удаляются, неизвестные блоки пропускаются, новые дописываются в конец выключенными. Правила разрешения документа — предмет продукта, а не формата.

8.4.9 Шаблон без блоков: цельная раскладка

Шаблон, не объявивший ни одного варианта `report.block` и ни одного состава в `reportDocument`, печатает свою раскладку вида `report` целиком — ровно как до версии 2.0.0. Ничего делать не нужно: это не режим совместимости, который включают, а естественное следствие правила «нет объявленного состава — нет документа из блоков».

Проверяется именно ОБЪЯВЛЕНИЕ СОСТАВА, а не наличие вариантов блоков: шаблон, переведённый наполовину, у непереведённого вида обязан печатать свой прежний макет, а не пустой документ.

---

9. DSL браузерного рендерера шаблонов

Браузерный рендерер поддерживает минимальный path-only DSL.

Поддерживается:

```
{{ path }}

{{#if path}}
  ...
{{/if}}

{{#unless path}}
  ...
{{/unless}}

{{#each path}}
  ...
{{/each}}

{{> partialName }}

{{& path }}
```

Не поддерживается:

- JavaScript;
- helper-функции;
- выражения внутри `if`;
- `{{{ path }}}` — у контролируемого HTML одна форма записи, `{{& path }}`.

Весь вывод `{{ path }}` экранируется как текст.

Контролируемый HTML: `{{& path }}`

`{{& path }}` печатает значение РАЗМЕТКОЙ, без экранирования. Это единственный канал разметки, доступный внутри `{{#each}}`: `data-placeholder` и `data-slot` адресуются по имени и повторяющемуся узлу не годятся, а именно в циклах печатаются толкования уровней и тексты

рекомендаций, которым автор теста задал формат «Форматированный» или «HTML» (`feedback_json.format`, см. §10.4).

Что попадает в такие поля, решает ЯДРО, а не автор макета: контекст несёт готовую разметку в парном поле (`textHtml`, `textsHtml`), которую ядро строит из текста и его формата. Обычный текст ядро экранирует само и переводит переводы строк в `<br>`, поэтому `{{& … }}` над полем `*Html` безопасен ровно в той же мере, в какой безопасен `richText` / `html` контентной страницы: источник один и тот же — автор теста.

Применять `{{& … }}` к произвольному полю контекста НЕЛЬЗЯ: поля, не построенные ядром как разметка, печатаются через `{{ … }}`.

HTML/rich content страницы по-прежнему вставляется через контролируемые placeholders/слоты, например:

```
data-placeholder="body"
data-slot="content-body"
data-slot="question-text"
data-slot="question-interaction"
data-slot="question-feedback"
```

Для контентных страниц предпочтителен `data-placeholder`, потому что он связан со схемой `contentTemplates[].placeholders[]`. `data-slot="content-body"` остаётся допустимым fallback для режима `html` и старых общих content layouts.

9.1 Контекст `each`

Внутри `each` текущий элемент становится текущим контекстом:

```
{{#each sections}}
  <button>{{ title }}</button>
{{/each}}
```

Корневой контекст доступен как:

```
{{ @root.test.title }}
```

Мета-переменные цикла:

```
@index   индекс с нуля
@number  индекс с единицы
@first   boolean
@last    boolean
```

9.2 Частичные шаблоны

Частичные шаблоны объявляются в `manifest.partials`:

```
{
  "partials": {
    "topicNav": "partials/topic-nav.html",
    "progress": "partials/progress.html"
  }
}
```

Использование:

```
{{> topicNav }}
```

Частичные шаблоны получают тот же публичный контекст, что и текущий макет.

Если частичный шаблон `topicNav` отсутствует, Core может использовать стандартный рендерер навигации по темам как резервный вариант.

---

10. Публичный контекст рендера

Все макеты получают общий публичный контекст.

> Типизированный контракт контекста рендера — `PublicRenderContext` с неймспейсами `course`, `state`, `result`, `retake`, `transition`, `design`, `review`, `sectionResult`, `sectionIntro`; страница отчёта получает сверх того неймспейс `report` (§10.2). Именно против него резолвятся `{{ path }}` и `data-path` в макетах (`data-path="course.title"`, `{{ result.scorePercent }}`, `{{#if state.canResume}}`). Перечень полей каждого неймспейса — в §7 и в разделах ниже. Пути `progress.*` доступны `resultField`-плейсхолдерам только в SCORM-рантайме (резолв против внутреннего `TEST_DATA`); на веб-хосте `resultField` резолвится против `PublicRenderContext`, где `progress.*` как топ-уровневого пути нет (есть `result.*` и `sectionResult.*`).

> Контентная страница дополнительно получает неймспейс `page.*`, вычисленный ядром из структуры теста: `page.dots` — точки индикатора последовательности (по одной на страницу отрезка, у текущей выставлен признак активной), `page.dotIndex` и `page.dotsTotal` — позиция и размер отрезка, `page.canGoBack` — доступен ли возврат на предыдущий экран в текущем режиме прохождения, `page.nextLabel` — подпись кнопки «вперёд» (настройка `nextLabel` страницы, по умолчанию «Далее»), `page.sectionSubtitle` — подзаголовок раздела на экране «Введение раздела»: строка над авторской инструкцией (настройки `sectionSubtitleShown` и `sectionSubtitle` варианта вида `intro`, по умолчанию «Инструкция»). Страница вне последовательности получает пустой набор точек, поэтому макет с индикатором деградирует до простой страницы. Рисовать индикатор и кнопку возврата — решение МАКЕТА; данные ядро отдаёт всегда.
> `page.dots/dotIndex/dotsTotal/canGoBack` автор не вводит; `page.nextLabel` и `page.sectionSubtitle` он задаёт настройками варианта. Все три хоста (SCORM-пакет, веб-хост, предпросмотр) читают `page.*` из одного общего ядра (`shared/template/page-sequences.ts`), так что считают одинаково.
>
> Две подписи различаются правилом пустоты, и это НЕ произвол. `page.nextLabel` пустым не бывает: кнопка без подписи — дефект, поэтому очищенное поле возвращает «Далее». `page.sectionSubtitle` пустым бывает — так автор убирает надпись, — но приходит это ТОЛЬКО от выключенного тумблера: пустое поле при включённом тумблере означает «следовать шаблону». Причина в редакторе: контрол настройки показывает объявленное умолчание серой подсказкой, и «не трогал» с «очистил» выглядят в форме одинаково, поэтому пустота не может значить «выключить». Макет обязан печатать надпись по условию:
>
> ```
> {{#if page.sectionSubtitle}}{{ page.sectionSubtitle }}
> {{/if}}
> ```
>
> Блок инструкции при выключенной надписи остаётся: гасится строка, а не авторский текст.

Пример ранней (нереализованной) формы контекста:

```
{
  "test": {
    "id": "test-1",
    "title": "Сертификация",
    "description": "",
    "navigationPolicy": "linear"
  },
  "page": {
    "id": "q-1",
    "type": "question",
    "kind": "question.single",
    "title": "Вопрос 1",
    "question": {
      "id": "q-1",
      "type": "single",
      "media": null
    },
    "answerState": {
      "hasAnswer": false,
      "locked": false,
      "feedbackVisible": false
    },
    "feedback": null
  },
  "sections": [
    {
      "id": "topic-1",
      "title": "Тема 1",
      "isActive": true,
      "isPassed": false,
      "className": "is-active"
    }
  ],
  "progress": {
    "active": {
      "current": 3,
      "total": 20,
      "percent": 15
    },
    "question": {
      "current": 3,
      "total": 20,
```

```
      "percent": 15
    },
    "page": {
      "current": 6,
      "total": 28,
      "percent": 21
    }
  },
  "nav": {
    "mode": "linear",
    "canPrev": true,
    "canNext": false,
    "canSubmitAnswer": true,
    "canFinish": false,
    "nextLabel": "Далее",
    "submitAnswerLabel": "Принять ответ",
    "finishLabel": "Завершить тест",
    "nextClassName": "is-disabled"
  },
  "params": {
    "brand.primaryColor": "#0066cc",
    "progress.mode": "questions"
  },
  "assets": {},
  "runtime": {
    "templateApiVersion": "1.0"
  }
}
```


Core подготавливает готовые классы состояния (например `passClass`, `statusLabel`); DSL шаблона классы через выражения не вычисляет.

10.1 Обратная связь и правильные ответы

Core раскрывает данные правильных ответов в публичном контексте только тогда, когда текущее состояние Core и настройки теста это разрешают.

До обратной связи:

```
{
  "answerState": {
    "hasAnswer": false,
    "locked": false,
    "feedbackVisible": false
  },
  "feedback": null
}
```

После отправки ответа, если правильные ответы можно показывать:

```
{
  "answerState": {
    "locked": true,
    "feedbackVisible": true,
    "scoreRatio": 0.5,
    "status": "partial"
  },
  "feedback": {
    "text": "Частично правильно",
    "correctAnswerPublic": {}
  }
}
```

Шаблоны не получают поддерживаемого прямого доступа к внутреннему `TEST_DATA`.

10.2 Контекст отчёта: `report.*`

Страница отчёта (§8.4) получает `course.*`, `design.*` и `result.*` в ТОЧНО ТОМ ЖЕ виде, что экран результатов, плюс собственный неймспейс `report.*`. Контракт публичный и расширяется только аддитивно: на него опираются макеты внешних шаблонов.

`result.*` строится ТЕМ ЖЕ построителем, что и экран результатов. Отчёт не вправе показать иной вердикт, чем экран, с которого его скачали, и не считает ничего самостоятельно — два независимых расчёта одного вердикта всегда расходятся.

Поле	Описание
`attemptDateLabel`	Готовая подпись даты прохождения (`дд.мм.гггг чч:мм`)
`attemptsCountLabel`	Готовая подпись числа попыток, со склонением; ПУСТА у теста, который ничего не оценивает, и у теста, где автор выключил «Показывать число попыток» (макет гейтит строку)
`learnerName` / `hasLearnerName`	ФИО слушателя (`cmi.learner_name` в LMS, пользователь сессии в вебе) и гейт строки: имя может быть неизвестно
`gridColumns`	Число колонок сетки тем, вычисленное ядром
`verdictHeadline` / `verdictBadge` / `verdictClass`	Заголовок, подпись бейджа и класс вердикта (`is-pass` / `is-fail`)
`correctLabel` / `earnedPointsLabel`	Готовые подписи «верно из общего числа» и заработанных баллов
`ringDasharray` / `ringDashoffset`	Геометрия дуги отчёта (своя, отличная от кольца экрана итогов)
`hasTopics`	Гейт блока тем
`courses[]` / `hasCourses` , `events[]` / `hasEvents`	Рекомендованные материалы и мероприятия: в обычном режиме — по проваленным темам без дублей, в адаптивном — по всем темам, у которых они есть, с названием темы
`isPreview`	Признак предпросмотра — макет может показать пометку «образец»
`values.<key>`	Значения `settings[]` варианта (§8.4.1). Поля типа `image` приходят готовой строкой: путь файла шаблона разрешён против базы хоста и инлайнен в data-URL (§8.4.1.1). Пустая строка = картинки нет, макет гейтит строку на ней

Строки `result.topicResults[]` в отчёте несут дополнительные ГОТОВЫЕ подписи, которых на экране нет: в обычном режиме — `verdictLabel` , `barPercent` , `countsLabel` , `pointsFixedLabel` ; в адаптивном — `hasCounts` , `answeredLabel` , `correctLabel` , `achievedClass` . Строки разреза (`breakdown[]` , §10.6) приходят в отчёт в том же виде, что и на экран: настройка показа одна.

Текста обратной связи темы среди них НЕТ: он печатается один раз, в консолидированном блоке `result.recommendations` . Прежний признак `showFeedback` снят вместе со слотом в карточке темы. Исключение — адаптивный отчёт: там `feedback` / `hasFeedback` строки темы означают обратную связь ДОСТИГНУТОГО УРОВНЯ (либо текст провала темы), в консолидированный блок она не подаётся, и слот в карточке сохранён.

Правило §10 действует и здесь: DSL не считает. Проценты, смещение дуги, число колонок, склонения и даты приходят готовыми.

10.3 Вводный блок: `result.introHtml`

Авторский текст, который идёт ПЕРВЫМ — до сводки баллов, результатов по темам, измерений и рекомендаций. Объясняет слушателю, что он сейчас читает.

Приходит РАЗМЕТКОЙ, уже построенной ядром из текста и его формата (§10.4), поэтому макет печатает его через `{{& … }}`:

```
{{#if result.introHtml}}<div class="tb-intro">{{& result.introHtml }}</div>{{/if}}
```

Пусто или поля нет = блока нет; второго признака у него не заведено. У экрана итогов и у отчёта тексты РАЗНЫЕ (их задают раздельно), но имя поля одно: макет печатает «свой вводный блок», не зная, чей контекст ему дали.

10.4 Авторский текст и его формат

Тексты, которые пишет автор ТЕСТА — обратная связь теста, толкование уровня шкалы, исход показателя, — приходят в контекст ПАРАМИ: строка и её разметка.

Поле	Что несёт
`result.recommendations.texts[]`	Тексты консолидированного блока строками
`result.recommendations.textsHtml[]`	Они же разметкой, индекс в индекс
`result.scales[].text` / `result.indicators[].text`	Толкование уровня строкой
`result.scales[].textHtml` / `result.indicators[].textHtml`	Оно же разметкой
`course.description`	Описание теста строкой (стартовый экран)
`course.descriptionHtml`	Оно же разметкой

Разметку строит ядро из текста и его формата (`feedback_json.format`: `plain`, `richText`, `html`). Обычный текст экранируется, а переводы строк становятся `<br>` — абзацы автора доходят до слушателя; `richText` и `html` печатаются как есть. Политика та же, что у полей контентной страницы (§8.2.1), и источник тот же — автор теста.

Макет печатает поле `*Html` через `{{& … }}` (§9), а парную строку — через `{{ … }}`. Шаблон, который печатает только строку, остаётся рабочим: он покажет текст без форматирования, как показывал всегда.

Гейт блока стоит на СТРОКЕ (`{{#if text}}`), а не на разметке: пустая строка и пустая разметка появляются вместе, но признаком наличия текста остаётся сам текст.

Описание теста на стартовом экране устроено так же. Формат берётся из своей колонки (`tests.description_format`), а не из `feedback_json`, но политика разметки та же:

```
{{#if course.description}}<div class="tb-cover__desc">{{& course.descriptionHtml }}</div>
{{/if}}
```

Элемент здесь блочный (`div`, не `p`): размеченное описание может нести абзацы и списки, а `<p>` внутри `<p>` браузер закрывает сам. И `data-path` на этом узле стоять НЕ должен — проход по `[data-path]` заполняет узел присваиванием `textContent` уже после прохода DSL и затёр бы разметку, которую интерполяция только что напечатала.

10.5 Надписи итогов и состав подблоков: `labels.*`, `result.blocks`

Экран итогов, адаптивные итоги, итоги раздела и отчёт получают в контексте дерево `labels.*` — уже РАЗРЕШЁННЫЕ тексты всех надписей, объявленных шаблоном (§8.3.1), в форме, которую адресует DSL. Пустая строка означает «не печатать»: макет гейтит блок на самом значении надписи, а не на отдельном флаге видимости.

```
{{#if labels.results.scales}}<h3 class="tb-scene__subhead">{{ labels.results.scales }}</h3>
{{/if}}
```

Дерево строит `labelsTree` (`shared/template/labels.ts`) из плоской карты «ключ → текст», которую отдаёт `resolveLabels`: путь `results.scales` разбивается по точкам, потому что `{{ labels.results.scales }}` резолвится обходом вложенных объектов, а не поиском по буквальной строке ключа. Ключ, который шаблон не объявил или который конфликтует с другим (взаимный префикс, отклоняется валидацией §17.1, но старый пакет мог быть собран до её введения), в дерево не попадает — макет, обратившийся к нему, получает пустую строку. Правило то же, что и везде в этом разделе: сборка контекста ТОТАЛЬНА, экран ученика не имеет права упасть из-за объявления шаблона.

`result.blocks[]` — состав и порядок подблоков итогов (§8.3.3), уже отобранных и упорядоченных ядром. Каждый элемент несёт `key`, эффективный `heading` (пустая строка — заголовок выключен, сам блок при этом остаётся) и ОДИН булев флаг вида (`isSummary` / `isScales` / `isIndicators` / `isTopics` / `isBreakdown`) — DSL не умеет сравнивать строки (§9), поэтому выбор ветки шаблону нужен уже готовым. Макет обходит массив ОДНИМ циклом вместо пяти фиксированных секций:

```
{{#each result.blocks}}
  {{#if heading}}<h3 class="tb-scene__subhead">{{ heading }}</h3>{{/if}}
  {{#if isScales}} … {{/if}}
{{/each}}
```

Видимость подблока (входит он в `result.blocks` вообще) и его заголовок (пуст ли `heading`) — независимые решения: скрытый подблок уносит заголовок с собой, а погашенный заголовок подблок не скрывает. Зонтичный заголовок (`labels.results.heading`) печатается только тогда, когда `result.blocks` непусто — видимых подблоков нет, печатать нечего.

10.6 Разрез результата: `result.topicResults[].breakdown`

Разрез — подытог попытки по ключу оси в пределах области (PRD-50). Ось — способ извлечь из выданного вопроса ключи; сейчас зарегистрирована одна, `tag` (подтемы вопроса). Область строки темы — РАЗДЕЛ ВЫДАЧИ: один ключ, заведённый в двух разделах, даёт в каждом свою запись.

Строка темы получает необязательный массив `breakdown[]` — уже подготовленные ядром строки:

Поле	Описание
`key`	Ключ оси — то, что печатается подписью строки (название подтемы)
`items` / `answered`	Сколько выданных вопросов несут ключ и сколько из них отвечено
`earned` / `possible`	Заработанные и возможные баллы по ключу, округлены до одного знака
`percent`	Величина ВЫБРАННОЙ автором базы, один знак после запятой — неокруглённый двойник `barPercent`. Макет, которому нужно число, печатает его и может не знать, какая база в силе
`percentUnits` / `percentPoints`	Нормированная доля (вес вопроса = 1) и балльная доля, по одному знаку. Балльная — валюта вердикта
`barPercent`	Ширина полосы в процентах, ОКРУГЛЕНА до целого; макет подставляет её в стиль
`showValue`	Печатать ли число рядом с полосой — гейт этого элемента разметки
`valueLabel`	Готовая подпись значения («50 %»); пуста при `showValue: false`
`passed`	Исход подтемы: `true` / `false` / `null` — порога у ключа не было
`passClass`	Готовый модификатор строки: `is-pass`, `is-fail` или пустая строка. Пуст и при окраске «По доле» или «Нейтральная» (см. ниже)
`barFill`	Готовая заливка полосы — значение CSS `background` вида `linear-gradient( … )`. Приходит только при окраске «По доле» и непустой полосе (с 3.12.0)
`requiredLabel`	Готовая надпись порога («Нужно 70 %»). Поля НЕТ, когда порога нет или автор не включил показ значения

Числа лежат в строке С САМОГО НАЧАЛА, даже когда поставляемые шаблоны печатают одну полосу: сторонний шаблон не может показать «верно 4 из 7», если чисел в контексте нет, а добавление поля позже — правка контракта, которую придётся возить по обоим хостам и по всем уже собранным пакетам. Форматирование при этом готовит ЯДРО (`showValue` , `valueLabel`), чтобы раскладка не решала его сама.

Исход в строке ЕСТЬ, словесной метки — нет (PRD-50 §16). С версии 3.1.0 строка снова несёт `passed` и `passClass` , а рядом — `requiredLabel` . Версия 3.0.0 снимала весь набор целиком; вернулась только та его часть, что говорит ТОНОМ. `statusLabel` не вернулся и не вернётся: строка узкая, и слово «Не пройдено» рядом с процентом дублировало бы цвет, а вердикт СЛОВОМ выносит ТЕМА, чья карточка стоит вокруг строки.

Красится ровно одна вещь — полоса. Порога у ключа нет — `passed` равен `null` , `passClass` пуст, `requiredLabel` отсутствует, и строка печатается ровно так, как печаталась до 3.1.0. Надпись порога приходит только там, где автор включил показ ЗНАЧЕНИЯ: цвет без числа и без причины читается как приговор.

Исход строка ПОЛУЧАЕТ у записи разреза, а не вычисляет сама. Иначе о пороге завелась бы вторая правда, и старая попытка перекрашивалась бы при каждой правке настроек теста.

Окраска полосы — выбор автора теста (с 3.12.0). Параметр оформления `breakdownBarFill` (тип `select` , §6) принимает три значения:

Значение	Что кодирует цвет	Что получает строка
`verdict`	Исход подтемы	`passClass` = `is-pass` / `is-fail` ; цвет задаёт CSS шаблона. Так же ведёт себя строка, когда параметр не объявлен или не задан
`share`	Долю	`passClass` пуст, `barFill` — градиент от неблагоприятного края схемы уровней к её цвету в точке доли
`neutral`	Ничего	`passClass` пуст, `barFill` нет; полоса нейтрального цвета шаблона

Градиент строится по той же рампе, что окрашивает уровни шкал и показателей (`levelScheme` и три цвета своей схемы): одна палитра «плохо → хорошо» на весь тест. Заливка считается от ширины САМОЙ полосы: полоса шириной `p` показывает ровно левую часть градиента, растянутого на всю дорожку, поэтому короткая полоса не заканчивается благоприятным цветом. Переход между цветами — прямой, в RGB, как у обычного CSS-градиента: так полосы печатались в отчёте и до 3.12.0. Зоны уровней идут по дуге оттенка и для пары «оранжевый → бирюзовый» проходят через жёлтый и зелёный; у полосы этого нет. Единственная промежуточная точка — середина схемы, если она задана и полоса её переходит. Цвета выводятся как `rgb( … )` только из тех троек HSL, которые ядро разобрало: испорченное значение своей схемы заменяется краем светофора, а не печатается.

Макет печатает заливку в `style` той же полосы, после ширины: инлайн-значение перебивает цвет заливки из CSS шаблона, поэтому правил под режимы шаблону не нужно. Шаблон, который `barFill` не печатает, в режиме «По доле» покажет полосу своим нейтральным цветом — вердикт-класс снят. Параметр действует одинаково на экране итогов (в карточке темы и в сводном блоке) и в отчёте.

```
{{#if breakdown}}
<div class="tb-topic__breakdown">
  {{#each breakdown}}
  <div class="tb-breakdown__row {{ passClass }}" data-item="{{ key }}">
    <div class="tb-breakdown__name">{{ key }}</div>
    <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;{{#if barFill}}
background: {{ barFill }};{{/if}}"></span></div>
    {{#if showValue}}<div class="tb-breakdown__val">{{ valueLabel }}</div>{{/if}}
    {{#if requiredLabel}}<div class="tb-breakdown__req">{{ requiredLabel }}</div>{{/if}}
  </div>
  {{/each}}
</div>
{{/if}}
```

Правила контракта:

- **Поля нет вовсе**, когда автор теста показ не включил или в теме нет ни одного ключа. Пустой массив в DSL ложен, поэтому один гейт `{{#if breakdown}}` убирает блок целиком вместе с его заголовком — второго признака видимости не заведено.
- **Показ — настройка ТЕСТА, а не оформления**: три положения (не показывать / полоса / полоса и число) и база показа (доля вопросов либо доля баллов). Настройка одна на все поверхности: экран итогов и отчёт печатают одно и то же.
- **База показа не меняет вердикт.** Порог ключа, когда он задан, всегда сравнивается с БАЛЛЬНОЙ долей — той же валютой, что и порог раздела; переключение вида отображения меняет только число и длину полосы. Поэтому ни `percent`, ни баллы строкой не передаются: макет печатает то, что ядро уже выбрало.
- **Это список ключей, а не разложение темы на части.** Ключи не обязаны разбивать выборку: вопрос может нести несколько ключей или ни одного, и сумма строк не обязана сходиться с итогом темы. Подписывать блок как «состав раздела» нельзя.
- **Те же строки в отчёте.** `result.*` в отчёте строится тем же построителем (§10.2), поэтому `breakdown[]` приходит и туда; макет отчёта печатает его своей вёрсткой.
- Считает разрезы один общий модуль (`shared/breakdown/`), который едет в бандл `shared-runtime`, поэтому веб-хост и пакет SCORM печатают на одной попытке одно и то же.

Имена классов в примере принадлежат «Стандартному» шаблону; контрактом является только состав данных.

10.7 Блоки разделов: `result.topicGroups`, `result.ungroupedTopics`

Автор теста может собрать разделы в НАЗВАННЫЕ блоки — «Обязательная часть», «Дополнительная часть» — и экран итогов печатает карточки тем по блокам, каждый со своим счётчиком «пройдено N из M». Вложенность ровно одна: тест держит блоки, блок держит разделы, дерева нет.

Поле блока	Описание
`key`	Ключ блока, каким его завёл автор
`label`	Заголовок блока; может быть пустым, если автор его не заполнил
`topics`	Карточки тем блока в порядке выдачи — ТЕ ЖЕ объекты, что и в `topicResults`
`passedCount` / `totalCount`	Разделы блока с вердиктом «пройден» и разделы с любым ВЫНЕСЕННЫМ вердиктом (раздел без вердикта не считается)
`counterLabel`	Готовый счётчик («1 / 2»); макет ничего не вычисляет

Правила контракта:

- **Поля нет вовсе**, когда автор блоков не заводил или ни один раздел в них не попал. Шаблон, ничего не знающий о блоках, печатает плоский `result.topicResults`, как печатал.
- `topicResults` **остаётся ПОЛНЫМ** рядом с блоками — и сгруппированные карточки, и несгруппированные. Поэтому макет, печатающий блоки, гейтится на `topicGroups` и печатает `topicGroups` + `ungroupedTopics` ВМЕСТО плоского списка, иначе тема выйдет дважды.
- `ungroupedTopics` — карточки, не попавшие ни в один блок, в своём порядке; печатаются ПОСЛЕ всех блоков. Поле едет отдельно потому, что DSL не умеет фильтровать список: без него макет не отличил бы уже напечатанные карточки от оставшихся. Поля нет, когда блок нашёлся каждой карточке.

10.8 Сводный разрез по тесту: `result.breakdown`

Разрез из §10.6 живёт в пределах РАЗДЕЛА. Тот же ключ, заведённый в двух разделах, даёт две строки, и сложить их макет не вправе. Сводный блок отвечает на другой вопрос — «как ключ выглядит по всему тесту» — и приезжает отдельным подблоком итогов (`breakdown` в `result.blocks`, §8.3.3), одной строкой на ключ.

Строки — того же типа, что и в карточке темы (таблица §10.6), поэтому вёрстка блока повторяет вёрстку строки. Считает их ядро отдельным проходом по выданному (`shared/breakdown/`), а не суммированием строк тем: сумма дала бы другое — и неверное — число, ради чего блок и заведён.

```
{{#if isBreakdown}}
<div class="tb-breakdown">
  {{#each @root.result.breakdown}}
  <div class="tb-breakdown__row" data-item="{{ key }}">
    <div class="tb-breakdown__name">{{ key }}</div>
    <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;{{#if barFill}}
background: {{ barFill }};{{/if}}"></span></div>
    {{#if showValue}}<div class="tb-breakdown__val">{{ valueLabel }}</div>{{/if}}
  </div>
  {{/each}}
</div>
{{/if}}
```

Правила контракта:

- **Где печатать — выбор автора теста**, третьим полем настройки показа: в карточках тем, сводным блоком или и там, и там. Поля нет в настройке, сохранённой раньше, — она читается как «в карточках тем», то есть ровно то, что тест печатал до появления блока.
- **Поля нет вовсе**, когда блок не выбран или попытка не дала записей области теста.
- **Заголовок блока** — надпись `results.breakdown` (§8.3.1), умолчание «Разрез результата»; позиция блока — `resultsBlockOrder` (§8.3.3), в поставляемом порядке он последний.
- **Адаптивный режим печатает ТОЛЬКО сводный блок.** Карточка темы там говорит подтверждённым уровнем и полос не несёт (§10.6 к ней неприменим), а сводный блок считается по тому же выданному и печатается и на экране, и в отчёте.
- **Те же строки в отчёте** — `result.*` отчёта строит тот же построитель (§10.2).

10.9 Лимит времени: `course.timeLimitLabel`

Лимит времени теста приходит в контекст ПАРОЙ: число минут и готовая надпись.

Поле	Что несёт
`course.timeLimitMinutes`	Лимит числом минут, как его задал автор
`course.timeLimitLabel`	Он же надписью: «14 дней», «1 день 2 ч», «2 ч 30 мин», «45 мин»

Надпись строит ядро (`shared/template/duration`): раскладывает минуты в дни, часы и минуты, опускает пустые части и склоняет «день». Единица измерения принадлежит ЯДРУ, а не макету, потому что движок намеренно не понимает ни выражений, ни хелперов (§9): макет, печатающий `{{ course.timeLimitMinutes }} мин`, на двухнедельном бюджете скажет «20160 мин» и другого сказать не сможет.

Печатать надо надпись, и гейт блока ставить на неё же — она пуста ровно тогда, когда лимита нет:

```
{{#if course.timeLimitLabel}}
<div class="tb-facts__item">
  <div class="tb-facts__num" data-path="course.timeLimitLabel"></div>
  <div class="tb-facts__lbl">на прохождение</div>
</div>
{{/if}}
```

Число остаётся в контексте: шаблон, связывающий только его, работает как прежде. Той же надписью печатается лимит РАЗДЕЛА (`sectionIntro.timeLimitLabel` , §7), поэтому обложка и введение раздела одного теста не могут разойтись в написании.

Бегущий отсчёт в шапке (`#timer-display` , `#section-timer-display`) считает и печатает рантайм, а не макет; с этой версии он растёт единицами так же — `M:SS` , `H:MM:SS` от часа и `D д H:MM:SS` от суток.

С версии 3.13.0 рядом стоит признак `course.closesOnLeave` (PRD-67 «Закрывать раздел при выходе»): `true` , когда выход из начатого раздела закрывает его навсегда, а в тесте без разделов — завершает попытку. Признак истинен, только если автор включил настройку И у теста есть хоть один лимит — общий или раздела; решает ОДНО общее правило (`testClosesOnLeave`), которое вызывают оба хоста. Приходит на стартовый экран, чтобы макет мог предупредить ДО начала:

```
{{#if course.closesOnLeave}}
<p class="tb-note">Выход из раздела закроет его: вернуться и дорешать будет нельзя.</p>
{{/if}}
```

Поставляемые макеты признак не печатают: оба хоста предупреждают ученика собственными уведомлениями ядра на первом вопросе раздела. У теста без настройки поля в контексте нет вовсе.

10.10 Толкование темы и подтемы: `interpretationHtml`

Толкование — текст, который ОБЪЯСНЯЕТ результат: что проверяет тема или подтема и что стоит за полученной оценкой. От обратной связи оно отличается не тоном, а правилом выдачи, и это различие — предмет контракта:

	Толкование	Обратная связь
Когда печатается	ВСЕГДА, при любом вердикте и при его отсутствии	по своему правилу: тема не взята; доля подтемы ниже общего порога теста
Где стоит	ПРИ своей теме или полосе	в сводном блоке «Рекомендации», вместе с курсами и материалами
Что несёт	только текст	текст, курсы, материалы, мероприятия

Поля контекста:

Поле	Что несёт
`result.topicResults[].interpretationHtml`	Готовая разметка толкования ТЕМЫ
`result.topicResults[].breakdown[].interpretationHtml`	То же для строки разреза (§10.6)
`result.topicResults[].hasBreakdownNotes`	`true`, когда хоть одна строка разреза несёт своё толкование
`result.topicResults[].hasAside`	`true`, когда у темы есть что показать ВТОРОЙ КОЛОНКОЙ: толкование либо строки разреза

Разметку строит ядро — тем же правилом, что и остальной авторский текст (§10.4), — поэтому печатать её надо через `{{& … }}`, а не через `data-path`: проход по `[data-path]` заполняет узел через `textContent` и затёр бы абзацы.

Правила контракта:

- **Поля нет вовсе, когда текста нет.** Пустая строка заставила бы макет печатать пустой блок, поэтому гейт стоит на самом поле.
- **Текст ТЕСТА заменяет текст ТЕМЫ целиком**, а не дополняется им. Разрешает это ядро, и макет получает уже разрешённое значение: две редакции одного текста, склеенные в выдаче, автор нигде не видит, а участник читает противоречие.
- **В блок «Рекомендации» толкование не попадает** ни при каких настройках: иначе одна и та же мысль напечаталась бы на экране дважды.
- **Толкования подтем показываются по настройке ТЕСТА** («Показывать толкование подтем»). Выключенная настройка не стирает текстов — они просто не приезжают в контекст.
- **Два признака раскладки** (`hasBreakdownNotes`, `hasAside`) считает ядро, потому что движок не умеет ни перебрать строки, ни сложить два вопроса в один ответ (§9). Макет, который их не связывает, работает как прежде.

10.11 Заголовки итога: свойства варианта «Итоги теста»

Тест вправе назвать свой итог сам — тремя свойствами варианта `results.*` (`settings[]`, §6). Это НЕ надписи словаря (§8.3.1): словарь переименовывает надписи интерфейса, общие для всех тестов шаблона, а здесь — содержание конкретного теста.

Ключ настройки	На что действует	Пусто означает
`headingDocument`	`course.title` на экране итогов и в отчёте	название теста
`headingPassed`	`result.statusLabel` на экране, `report.verdictHeadline` в отчёте	«Пройден» / «Тест пройден»
`headingFailed`	там же при неуспехе	«Не пройден» / «Тест не пройден»

Правила контракта:

- **Новых полей контекста нет.** Значения приходят в те же поля, что и раньше, — макет правок не требует.
- **Умолчания у экрана и у документа РАЗНЫЕ** и остаются за ядром: над шапкой документа нет названия теста, и одним словом она не держится.
- **Тест без вердикта заголовками исхода не переименовывается**: они утверждали бы о слушателе то, чего экран не утверждает (§7.3). Заголовок документа при этом действует — как назван итог, от вердикта не зависит.
- **Шаблон, не объявивший этих ключей, полей автору не покажет**, и тест на нём останется с прежними умолчаниями. Объявление — обязанность шаблона, как и у любой другой настройки варианта.

10.12 Условие прохождения: обложка и введение раздела

С версии 3.14.0 условие прохождения называется ученику ДО первого ответа — на обложке и на экране «Введение раздела». Формулировки строит ОДИН модуль ядра (`shared/template/pass-condition`) по тем же правилам, по которым вердикт выносит `aggregateStandardResult` : тема судится, когда её разрешённое правило не пусто; отсутствующий признак `required` означает «обязательная»; порог «количество» сравнивается с суммой набранных баллов. Экран не вправе обещать условие, которого судья не применит.

Обложка получает `course.passCondition` , когда исход решают темы (политика «Тест пройден, если», `tests.pass_decision_policy`):

Политика	`course.passCondition`	`course.passPercent`
Только общий результат	нет	есть
Общий результат и обязательные темы	обязательные темы с порогом, `kind: "required"`	есть
Все обязательные темы	обязательные темы с порогом, `kind: "required"`	нет (справочный)
Все темы	все темы с порогом, `kind: "all"`	нет (справочный)

Поля объекта: `count` и `total` (равны — пройти надо все темы этого рода), `kind`, `countLabel` — число плитки («8 из 8») и `label` — подпись («обязательных тем для прохождения» / «тем для прохождения»). Поля нет, если тем с порогом нет, у измерительной методики и у адаптивного теста. Плитка печатается рядом с остальными фактами обложки:

```
{{#if course.passCondition}}
<div class="tb-facts__item">
  <div class="tb-facts__num" data-path="course.passCondition.countLabel"></div>
  <div class="tb-facts__lbl" data-path="course.passCondition.label"></div>
</div>
{{/if}}
```

Введение раздела получает три поля `sectionIntro.*`:

Поле	Тип	Смысл
`passCondition`	string	Порог темы готовой строкой: «Для прохождения: 70 %» или «Для прохождения: 7 баллов из 10». Порог берётся по ВЫДАННОМУ варианту (PRD-24), «из M» — сумма цен выданных оцениваемых вопросов раздела. Пустая строка, если тема не судится или в ней нечего оценивать
`isRequired`	boolean	Тема обязательна для вердикта — метка «Обязательная тема». Истинно только при политиках «Общий результат и обязательные темы» и «Все обязательные темы» и только у темы с порогом
`timerWarning`	string	Системное предупреждение, когда у раздела свой лимит времени: отсчёт начнётся по кнопке продолжения и паузы не будет. Пустая строка без лимита

Отсчёт раздела запускается на ПЕРВОМ вопросе раздела в обоих хостах, поэтому текст предупреждения правдив: пока ученик читает введение, время раздела не идёт. Поставляемые шаблоны печатают предупреждение баннером ДС уровня `warning`; в центрированной колонке баннер сохраняет выравнивание текста по левому краю (`.tb-scene__col--center .ou-banner`).

Хосты строят поля одним построителем: веб получает правила разделов и цены выданных вопросов от сервера вместе с попыткой (`sectionConditions`), пакет вычисляет их из `TEST_DATA` и выданного варианта. Шаблон, не связывающий новых полей, показывает оба экрана как раньше.

С версии 3.15.0 строку условия можно убрать на конкретном введении раздела. Вариант вида `intro` объявляет тумблер страницы:

```
{ "key": "passConditionShown", "type": "boolean", "default": true,
  "label": "Показывать условие прохождения" }
```

Выключенный тумблер ядро разрешает в пустой `sectionIntro.passCondition` (`passConditionShownOf` в `shared/template/page-sequences`), поэтому макет ничего нового не связывает: тот же гейт `{{#if sectionIntro.passCondition}}` убирает строку. Гасит её только явное `false` : страница, сохранённая до появления настройки, и вариант, который её не объявил, печатают строку как раньше. Тумблер меняет только надпись — порог темы и вердикт судья выносит прежние, а `isRequired` и `timerWarning` от него не зависят.

10.13 Факты обложки: порог в баллах и тумблеры плиток

С версии 3.17.0 общий порог, который автор задал в БАЛЛАХ (`overall_pass_rule_json.type` — `absolute`), обложка называет в баллах. Судья сравнивает такой порог с суммой набранных баллов, а максимум баллов зависит от выданных вопросов и их цены, поэтому постоянного процента у него нет. Обложка получает `course.passPoints` , а `course.passPercent` остаётся пустым:

Поле	Тип	Смысл
`value`	number	Порог, как его задал автор
`valueLabel`	string	Число плитки: «5», «7,5»
`label`	string	Подпись, согласованная с числом: «балл / балла / баллов для прохождения»

Поле гасится там же, где `passPercent` : у измерительной методики и при политиках, где исход решают темы. У порога в процентах поля нет — контекст такого теста прежний. Плитка:

```
{{#if course.passPoints}}
<div class="tb-facts__item">
  <div class="tb-facts__num" data-path="course.passPoints.valueLabel"></div>
  <div class="tb-facts__lbl" data-path="course.passPoints.label"></div>
</div>
{{/if}}
```

Порог согласования балла с вердиктом, который пакет отправляет в LMS, считается так же по баллам: процентом от возможных баллов ЭТОЙ попытки (`lmsThresholdFor` в `shared/scoring/lms-score`). `TEST_DATA.passPercent` у порога в баллах — `null` .

Варианты вида `start` объявляют четыре тумблера страницы, по одному на плитку:

```
{ "key": "questionCountShown", "type": "boolean", "default": true, "label": "Показывать число вопросов" },
{ "key": "passConditionShown", "type": "boolean", "default": true, "label": "Показывать условие прохождения" },
{ "key": "timeLimitShown", "type": "boolean", "default": true, "label": "Показывать ограничение времени" },
{ "key": "maxAttemptsShown", "type": "boolean", "default": true, "label": "Показывать число попыток" }
```

Выключенный тумблер ядро разрешает в пустое поле (`hiddenStartFacts` и `hideStartFacts` в `shared/template/start-facts`): `questionCount` пропадает, `passPercent` пустеет вместе с `passPoints` и `passCondition` , `timeLimitMinutes` и `timeLimitLabel` пустеют, `maxAttempts` пустеет. Макету ничего нового связывать не нужно — плитку убирает её собственный гейт. Тумблер действует, только если вариант страницы его объявил, и гасит плитку только явное `false` . Подзаголовок «Попытка N из M», порог и вердикт от тумблеров не зависят. Хосты применяют одно правило: пакет читает свойства страницы из `TEST_DATA.contentPages` , веб получает список скрытых фактов с макетом старта (`hiddenFacts` в ответе `GET /screen-template/start`), предпросмотр в редакторе — из черновика страницы.

---

11. Навигация

Поддерживаемые режимы навигации:

```
linear
free
locked
```

Шаблон объявляет поддерживаемые режимы в `manifest.capabilities.navigation` .

Бизнес-политикой навигации владеет тест. Возможности шаблона только ограничивают доступные режимы.

Эффективный режим навигации:

```
effective mode = test navigationPolicy, ограниченная возможностями шаблона
```

Параметры шаблона могут управлять представлением навигации, но не должны ослаблять политику теста.

Пример:

```
{
  "test": {
    "navigationPolicy": "linear"
  },
  "params": {
    "navigation.presentation": "sidebar"
  }
}
```

---

12. Runtime API для `template.js`

`template.js` опционален и исполняется в браузере обучающегося.

Реализованный API:

```
TestBuilder.template.on(event, handler);   // события жизненного цикла
TestBuilder.template.emit(event, data);
TestBuilder.context.get();                 // { params } — эффективные значения параметров
TestBuilder.scorm.commit();                // фиксация SCORM-состояния
TestBuilder.ui.toast(message);             // заглушки: только console.warn
TestBuilder.ui.modal(options);
TestBuilder.renderers.register(id, fn);    // рендерер для resultField
TestBuilder.renderers.render(field, context, allowed);
```

Реально эмитируется одно событие — `page:enter` (на контентных страницах).

Вне охвата (логика теста задаётся сценарием, см. §14): `vars.*` (переменные времени прохождения), `nav.*` (программная навигация), `timer.*`, `ui.setState`, а также `scorm.setSuspendData` / `scorm.addInteraction`.

Прямой доступ к `window.API_1484_11` не входит в поддерживаемый контракт. Поддерживаемые SCORM-операции идут через `TestBuilder.scorm`.

---

13. Переменные времени прохождения — вне формата

Изменяемые переменные времени прохождения и их сохранение в `suspend_data` в формат шаблона не входят. Тест не оперирует произвольным мутабельным состоянием: логика прохождения задаётся декларативным сценарием (`flowPolicy`, см. §14), а вычисляемые результаты — это шкалы (`scale.*`) и показатели (`result.*`), которые Core публикует сам после расчёта результата. Шаблону в контексте рендера доступно пространство `result.*` с показателями (§10); собственного мутабельного состояния у шаблона нет.

14. Логика прохождения — вне формата

Логика прохождения теста задаётся декларативным сценарием, а не шаблоном и не универсальным движком правил. Ветвление и завершение выражаются структурными настройками сценария — `flowPolicy` (`flowMode`, `completionPolicy`, `sectionUnlockRules`, `passDecisionPolicy`), а условия по результату — показателями и адаптивными уровнями. Шаблон на эту логику не влияет: он отрисовывает экраны, которые ему передаёт Core.

---

15. Реестр шаблонов

Для встроенных и загруженных шаблонов используется один API реестра.

Концептуальные записи:

```
{
  "id": "corporate",
  "sourceType": "builtin",
  "sourcePath": "server/scorm/templates/corporate"
}
```

```
{
  "id": "rtk-custom",
  "sourceType": "uploaded",
  "sourcePath": "uploads/templates/rtk-custom"
}
```

Core использует:

```
TemplateRegistry.getTemplate(id)
```

и получает одну и ту же нормализованную файловую структуру независимо от физического источника.

---

16. Настройки дизайна теста

Рекомендуемая форма хранения:

```
{
  "templateId": "corporate",
  "templateVersion": "1.2.0",
  "templateApiVersion": "1.0",
  "params": {
    "brand.primaryColor": "#0066cc",
    "progress.mode": "questions"
  }
}
```

Строгое закрепление файловой версии не требуется. Сохранённые версии используются для диагностики, проверки совместимости и сообщений о миграции параметров.

Выбор варианта ОТЧЁТА и значения его полей (§8.4) хранятся ОТДЕЛЬНО от настроек дизайна:

```
{
  "standard": {
    "variantKey": "report.standard",
    "values": { "backgroundImage": { "url": "/uploads/media/bg.png" } }
  },
  "adaptive": {
    "variantKey": "report.adaptive.compact",
    "values": { "headline": "Итоги аттестации" }
  }
}
```

Ключи ветвей — РЕЖИМЫ теста, а не виды манифеста: тест одного режима заполняет одну ветку, но обе сохраняются, чтобы настройка не терялась при смене режима. Отсутствие ветки означает вариант `isDefault` активного шаблона со значениями `default` из манифеста, а отсутствие вида — деградацию по §8.4.4.

Тем же правилом настройка показа разреза (§10.6) — своя колонка теста, а не запись в настройках дизайна: она решает, что ученик увидит, а не как это выглядит, и правится на вкладке «Настройки».

Разнесение сделано намеренно, а не по недосмотру: настройки дизайна коммитятся черновиком вкладки «Оформление», тогда как поля отчёта автор правит в блоке обратной связи вкладки «Настройки». Общее хранилище связало бы две вкладки порядком сохранения. Следствие для реализации: смена шаблона может обнулить набор доступных вариантов, поэтому доступность пересчитывается на ЧЕРНОВОМ `templateId` и автор предупреждается до сохранения.

> **Реализация.** Настройки дизайна — `tests.design_settings_json`, выбор отчёта — `tests.report_settings_json` (обе колонки nullable). Публикация теста пиннит выбранный вариант и значения полей в снапшот вместе с остальным содержимым: отчёт по старой попытке собирается тем макетом и теми параметрами, что действовали на момент выдачи.

---

17. Валидация

17.1 Структурная валидация

Блокирующие ошибки:

Блокируют только нарушения целостности пакета и безопасности — то, при чём шаблон нечитаем или небезопасен и подставить вместо него нечего:

- невалидный ZIP;
- отсутствующий или невалидный `manifest.json`;
- отсутствующие обязательные поля манифеста;
- неподдерживаемый `templateApiVersion`;
- отсутствующие файлы, на которые ссылается манифест;
- отсутствующий или невалидный `assets.preview`;
- отсутствующий или невалидный `preview`;
- отсутствующий, невалидный или несовместимый `preview.demoData`;
- внешние URL в ресурсах/макетах/скриптах/стилях, на которые ссылается манифест;
- невалидный DSL в макете (незакрытый блок, `{{{ }}}`, выражение) — код `LAYOUT_TEMPLATE_SYNTAX`;
- невалидные `contentTemplates[]`: отсутствует `key`, `label` или `kind` (по схеме манифеста);
- неизвестный тип поля в `placeholders[]` или `settings[]`, а также несовместимые атрибуты типа (например `select` без списка вариантов);
- невалидные объявления вариантов отчёта (§8.4): вариант без `key` или без `layoutFile`, отсутствующий в пакете файл макета или `styleFile`, тип `sequence` либо непустой `placeholders[]` у ОБОЛОЧКИ отчёта, не ровно один `isDefault` на объявленный вид;
- невалидные объявления БЛОКОВ документа (§8.4.7): вариант `report.block` без `block`; неизвестный ключ блока (перечень закрыт продуктом); не ровно один `isDefault` на пару «блок + вид»; непустой `placeholders[]` у блока, отличного от `page`;
- невалидный `reportDocument` (§8.4.8): не карта «вид → список», неизвестный ключ блока в списке, повтор блока С ДАННЫМИ (повтор `page-break` ошибкой не является), блок в списке, у которого шаблон не объявил ни одного варианта;
- нарушение среды стилей отчёта (§8.4.3): макет без корневого класса `tb-report`, классы слоя сцены в макете отчёта, селектор `styleFile` вне `.tb-report` либо адресующий документ (`:root` / `html` / `body`);

- невалидное объявление надписей `labels[]` (§8.3.1): не массив, запись без `key`, дублирующийся `key`, надпись без `default`, взаимно-префиксные ключи;
- невалидные возможности (`capabilities`).

Предупреждения:

- неиспользуемые параметры;
- неиспользуемые ресурсы;
- отсутствующий макет — экран рендерится из стандартного шаблона (§8.2);
- отсутствующий слот, включая слоты вопросов и `page-content`;
- отсутствующие хуки/действия `shell.html`;
- route в `preview.routes`, для которого нет layout/template/capability;
- предупреждения консоли в браузерной smoke-проверке;
- fallback renderer в live preview, если страница осталась работоспособной;
- отсутствующее покрытие необязательного route в demo dataset;
- возможность объявлена, но опциональный partial отсутствует там, где существует резервный рендерер Core.

> **Реализация.** Фактические блокирующие коды структурного валидатора (`server/services/template-validation.ts`): `ZIP_TOO_LARGE`, `MANIFEST_MISSING`, `MANIFEST_INVALID_JSON`, `MANIFEST_SCHEMA`, `ID_PATTERN`, `API_VERSION_UNSUPPORTED`, `ID_EXISTS`, `ID_MISMATCH`, `REQUIRED_FIELD_MISSING` (только `assets.preview` / `preview` / `params` / `capabilities`), `FILE_MISSING`, `LAYOUT_TEMPLATE_SYNTAX`, `EXTERNAL_URL`, `DEMODATA_INVALID_JSON`, `REPORT_VARIANT_INVALID` (сообщение называет вариант и место замечания), `LABELS_INVALID` (сообщение называет индекс и ключ; функция — `validateLabelDeclarations`, §8.3.1). Проверки отчёта идут дважды и с разной глубиной: при ЗАГРУЗКЕ доступны файлы пакета, поэтому проверяются макет и CSS; активационный гейт файлов не открывает и проверяет только ОБЪЯВЛЕНИЕ (та же схема, что у проверки тем). Предупреждения: `SHELL_CONTRACT`, `QUESTION_CONTRACT`, `CONTENT_CONTRACT` (нет `data-slot="page-content"`), `OPTIONAL_LAYOUT_MISSING` (не объявлен макет экрана), `UNUSED_FILE`. Определения `params` и объявления `systemPages` схемно НЕ валидируются (проходят через `passthrough`); их наличие в списках выше — часть контракта.
>
> Проверка типов полей выполняется при ЗАГРУЗКЕ и при АКТИВАЦИИ шаблона: невалидный шаблон не принимается и не активируется, сообщение содержит ключ варианта и ключ поля. Ранее установленные шаблоны продолжают работать, но помечаются невалидными с указанием причины, а страница с полем неизвестного типа показывает в редакторе явную диагностику вместо подмены контрола.

17.2 Валидация макетов

Слоты, которые проверяет структурный валидатор (согласовано с §7 и §8.1). **Ни один из них не блокирует активацию**: их отсутствие даёт предупреждение и fallback экрана на стандартный шаблон (§17.1). Объявлять их всё равно следует — иначе шаблон отдаёт свой экран стандартному и теряет оформление.

Оболочка `shell.html` — область страницы (иначе `SHELL_CONTRACT`):

```
<div data-slot="page"></div>
```

Макет вопроса — два слота (иначе `QUESTION_CONTRACT`):

```
<div data-slot="question-text"></div>
<div data-slot="question-interaction"></div>
```

Контентная страница — слот `page-content` (иначе `CONTENT_CONTRACT`):

```
<div data-slot="page-content"></div>
```

Кнопки `data-nav="next"` / `data-action="answer-submit"` / `data-action="test-finish"` валидатор в оболочке **не требует** (Core привязывает их делегированием, когда они есть) — это относится к перспективным расширенным интерактивам (§3). Дополнительно валидатор компилирует каждый макет: невалидный DSL (незакрытый блок, `{{{ }}}`, выражение) даёт блокирующую ошибку `LAYOUT_TEMPLATE_SYNTAX` — единственная блокирующая проверка макета ЭКРАНА.

Макет ОТЧЁТА проверяется строже, и эти проверки блокирующие (§17.1):

```
<div class="tb-report"> … </div>          ←!— корневой класс обязателен —→
```

```
.tb-report__card { … }                    /* допустимо: селектор скоуплен */
:root { --brand: #000; }                  /* запрещено: адресует документ */
.tb-scene__footer { … }                   /* запрещено: слой сцены */
```

Строгость здесь не про вкус: несоответствие §8.4.3 в SCORM-пакете не проявляется и всплывает только в браузере, то есть на живой приёмке. Ревью такое не ловит — ловит только проверка.

17.3 Браузерная smoke-проверка

После структурной валидации запускается браузерная smoke-проверка на `preview.demoData`. Набор маршрутов берётся из `preview.routes` или строится Core из возможностей манифеста и должен покрывать:

```
стартовая страница
content.intro
content.info
question.single
question.multiple
question.matching
question.ranking
content.summary
results
system.locked, если объявлен
topicNav, если объявлен
progress.active
кнопки навигации/действий
загрузка template.js
```

Критерии успешного прохождения:

```
нет необработанных ошибок
next/answer-submit/test-finish привязаны
ответы сохранены в состояние Core
страница результатов открывается
```

Незаполненный слот критерием прохождения не является: он даёт предупреждение и fallback экрана на стандартный шаблон (§17.1).

> **Реализация.** Фактическая smoke-проверка рендерит каждый маршрут из `preview.routes` на демо-данных общим рендерером и помечает экран проваленным при исключении рендера, пустом результате или `console.error` (`console.warn` — предупреждение). Отсутствующий слот и необъявленный макет дают предупреждение: если хосту передан вход `fallbackLayouts`, экран рендерится из стандартного шаблона. Дополнительно компилируется `template.js`. Привязка `next/answer-submit/test-finish` критерием прохождения не является (относится к перспективным расширенным интерактивам, §3).

Сверх маршрутов `preview.routes` smoke-проверка рендерит ОТЧЁТ — по одному варианту каждого объявленного вида (§8.4). В `preview.routes` отчёт не значится: это не экран прохождения, а документ, — но ломается он так же, а замечает автор только тогда, когда обучающийся не смог скачать PDF. Ошибка отрисовки макета отчёта блокирует активацию наравне с ошибкой экрана.

> **Реализация.** Данные отчёта в проверке строит тот же построитель, что и предпросмотр в настройках теста, на структуре демонстрационного набора (`preview.demoData` : название и темы). Берётся исход «не пройден» — на нём страница показывает больше: вердикт, непройденные темы и блок рекомендаций, то есть отрисовывается больше макета.

Загрузка/обновление шаблона также должны предоставлять:

- статический предпросмотр из `assets.preview` ;
- живой предпросмотр на `preview.demoData` в админском UI.

Авторский UI может показывать статический предпросмотр в галерее и живой предпросмотр в деталях.

---

18. Обработка runtime-ошибок

SCORM-пакет включает минимальный экран ошибки Core, независимый от выбранного шаблона.

Если рендеринг шаблона или runtime шаблона падает после экспорта, Core показывает аварийный экран:

```
Не удалось отобразить страницу курса.
Код ошибки: TEMPLATE_RENDER_ERROR
Попробуйте обновить страницу или обратитесь к администратору.
```

Core должен логировать детали в:

- консоль браузера;
- диагностику `suspend_data` , где возможно;
- telemetry, если включена.

Пакет не включает стандартный шаблон целиком: в него добавляются только резервные макеты и стили стандартного шаблона для системных экранов и видов отчёта, не объявленных выбранным шаблоном.

19. Зона ответственности SCORM

Поддерживаемые SCORM-операции шаблона идут через `TestBuilder.scorm`.

Core владеет:

- стандартными полями score;
- completion/success status;
- стандартными интерактивами вопросов;
- публикацией пользовательских показателей результата;
- слиянием `suspend_data`;
- telemetry payload при завершении.

Шаблонный код (`template.js`) может запрашивать поддерживаемую SCORM-операцию:

```
TestBuilder.scorm.commit();
```